TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Background of Practical Training

Practical training is a compulsory course for all Diploma in Computer Science students and must be completed in order to fulfil the graduation requirements. This training provides students with relevant job experience related to the knowledge and skills they have acquired throughout previous semesters of study.

A duration of 24 weeks is allocated for Diploma in Computer Science students to undergo practical training at an organisation of their choice. Students may choose to undertake their training in either government agencies or private companies, provided that the scope of work is relevant to their field of study. Throughout the training period, students are required to perform assigned tasks under the supervision of experienced and professional personnel. Assessment of the practical training is based on students' performance and the submission of a comprehensive training report.

1.2 Objective of the Practical Training and the Report

The main objective of practical or industrial training is to provide students with hands-on working experience by performing specific tasks assigned by the organisation. In addition, the practical training aims to:

I. To build students confidence after graduation
II. Improve both soft and hard skills.
III. Improve communication and management skills.
IV. To expose the real world of the working environment to the students.

Upon completion of the practical training within the allocated period, students are required to prepare a written report in accordance with the standards set by the University. This report serves as an evaluation tool to assess the effectiveness of the students' learning and skills development during the training period. It also acts as documented evidence of the student's participation in practical training and may serve as a reference for future use. Furthermore, the preparation of this report helps students develop professionalism and awareness of workplace rules, regulations, and ethical practices.

CHAPTER 2: BACKGROUND OF THE ORGANIZATION

2.1 Introduction

Petroliam Nasional Berhad (PETRONAS), Malaysia's national oil and gas company established in 1974, has grown into an integrated multinational corporation operating across upstream, midstream, downstream, and gas and new energy segments. While upstream activities focus on exploration and production, the downstream segment involves refining, petrochemical production, and marketing and distribution of petroleum products. As part of its downstream operations, PETRONAS has several refining subsidiaries, including Malaysian Refining Company Sdn. Bhd. (MRCSB), located in Melaka, which processes crude oil into high-value products for domestic and international markets (refer to **Appendix A**).

As shown in the aerial view in **Figure 2.1**, MRCSB operates on a 926-acre refinery complex and refines up to 300,000 barrels of crude oil per day, supplying approximately 60% of Peninsular Malaysia's fuel demand. Its operations include major processing units such as PSR-1 (sweet crude), PSR-2 (sour crude), MG3 (lube base oil production), DELIMA (EURO-5 diesel), and the many supporting facilities like the COGEN plant for power and steam generation. These facilities include administration buildings, fire station (BOMBA), warehouses, and engineering workshops. The refinery continuously upgrades digitalisation, automation, and predictive maintenance across over 78,000 equipment units, ensuring reliability, safety, and operational excellence. Furthermore, sustainability initiatives, aligned with PETRONAS' Net Zero Carbon Emissions 2050 goal, include emission control systems, energy-efficient technologies, waste minimisation, and integrated cogeneration.

The refinery is divided into five main areas:

- **Areas 1 & 6 (PSR-1 – Sweet Hydroskimming):** 8 refining units for sweet crude and lube base oil production.

- **Area 2 (PSR-2 – Sour Hydroskimming):** 12 units for sour crude processing and product upgrading.

- **Area 3 (PSR-2 – Sour Conversion):** 7 units including hydrocracking and delayed coking.

- **Area 4 (Offsites):** Crude/product storage tanks, SBM terminal, and effluent treatment.

- **Area 5 (Utilities):** Steam, cooling water, plant air, nitrogen, amine treatment, and houses the COGEN plant.

***Figure 2.1** : Aerial View of MRCSB*

Figure 2.2 *: Asset Integrity Management Department (AIMD) Logo*

Represented by the official logo in **Figure 2.2***,* the Asset Integrity Management Division (AIMD) ensures the safe and reliable operation of all fixed equipment. Its core responsibilities include maintaining equipment integrity, minimizing operational hazards, and ensuring compliance with regulatory and internal standards, aligning with MRCSB's HSEMS. Furthermore, AIMD's objectives include optimizing operational and capital expenditures, managing risks to ALARP levels, and executing corrective actions to maintain safe operating conditions.

AIMD is organized into five main sections:

- **Material & Corrosion Engineering (M&C):** Damage identification and inspection strategy.

- **Manage Facility Integrity (MFI):** Plant inspection supervision and contractor monitoring.

- **Inspection Planning:** Develops and coordinates inspection plans.

- **Special Emphasis Integrity (SEI):** Handles high-priority and specialized inspections.

- **Assurance Self-Regulation (ASR):** Manages risk-based inspections and assurance processes.

Through these initiatives, AIMD contributes significantly to MRCSB's operational excellence, safety, and sustainability objectives.

2.2 Vision, Mission & Objectives of the Organization

Vision

To be a Leading Oil and Gas Multinational of Choice.

Mission

- Operate as a business entity with petroleum as its core business.
- Develop and add value to Malaysia’s national hydrocarbon resources.
- Contribute to the well-being of the people and the nation.

Objectives

- Ensure optimum value creation through its integrated business model by managing and maximising the nation’s hydrocarbon resources.
- Deliver energy and solutions that meet current and future energy needs responsibly and sustainably.
- Achieve long-term growth while supporting economic development and environmental stewardship.
- Support the transition to a lower carbon energy future and pursue energy innovation.

2.3 Organizational Chart

Malaysian Refining Company Sdn. Bhd. (MRCSB) is a large and complex organization with multiple divisions and subsidiaries. For the purpose of this report, the focus is on the Asset Integrity Management Division (AIMD), where the practical training was conducted; the organizational hierarchy of this division is detailed in **Figure 2.3**.

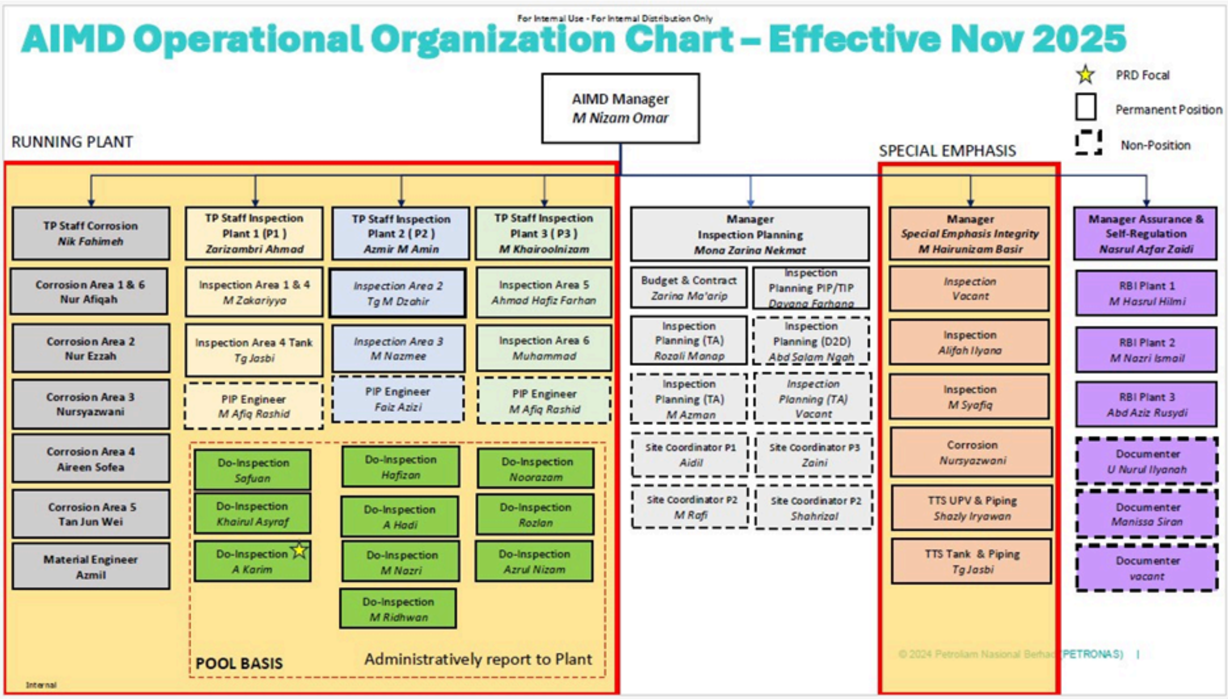


***Figure 2.3** : AIMD Organization Chart*

The chart illustrates the hierarchy and functional responsibilities within AIMD:

- **AIMD Manager:** Oversees the entire division, ensuring alignment with MRCSB's operational, safety, and integrity objectives.

- **Running Plant Section:** Comprises inspection and corrosion teams, including personnel responsible for various plant areas and Do-Inspection teams. These teams monitor the condition of fixed equipment and ensure compliance with safety standards.

- **Inspection Planning Section:** Handles the planning of inspections, budgeting, and coordination with site teams across different plants.

- **Special Emphasis Section:** Focuses on high-priority and specialized integrity tasks, such as certified fitness renewals, corrosion monitoring, and TTS (Tank & Piping) inspections.
- **Assurance & Self-Regulation Section:** Manages risk-based inspections (RBI) and assurance activities to maintain plant reliability and compliance with statutory requirements.

This chart highlights the functional structure and division of responsibilities within AIMD, emphasizing the collaborative approach required to maintain equipment integrity, operational reliability, and safety across MRCSB's refinery operations.

CHAPTER 3: MAIN PROJECT

3.1 Introduction

The main project undertaken during my practical training is VISTA (Visibility into Inspection Scheduling, Tracking & Assurance), a comprehensive digital platform designed to centralize inspection scheduling, tracking, and reporting within AIMD. VISTA aims to provide real-time visibility and improve operational efficiency by integrating multiple inspection-related activities into a single system.

While VISTA is a large-scale project with multiple modules, my focus was on the following:

- **AATLAS Module (Completed)** – Responsible for handling completed inspection emails, automating record updates, and reducing manual input errors.

- **CORE Module (Ongoing)** – Continuing development from past senior interns' work to track ongoing inspection activities, integrate workflow updates, and simplify reporting and approvals.

This project supports AIMD's initiative to digitize all inspection-related processes, addressing issues such as duplicated data entry, fragmented databases, and reliance on Excel for activity tracking. The development of these modules contributes to a more efficient, centralized, and transparent system for inspection scheduling and assurance.

3.2 Objectives

- To centralise inspection data management by replacing manual and scattered sources with a digital system.

- To develop backend and user interface functionality within a single integrated module.

- To improve coordination through real-time monitoring of inspection progress and task status.

3.3 Hardware & Software Requirements

3.3.1 OutSystems Service Studio Software

The main development tool used for the VISTA project is OutSystems Service Studio, a low-code platform designed for rapid development and deployment of web-based applications. OutSystems was chosen due to its ability to accelerate development, integrate multiple data sources, and simplify system maintenance, which was crucial for a large-scale project like VISTA.

Key features and their application in the project include:

- **Low-code development environment:**
  OutSystems allows building web-based modules with minimal manual coding. For the VISTA project, this enabled rapid development of the AATLAS and CORE modules, while ensuring the system remains maintainable for future enhancements.

- **Database integration:**
  The platform supports connections to multiple databases. This feature was used to consolidate fragmented AIMD data from more than 32 existing databases, forms, and dashboards into a centralized system, improving data integrity and reducing duplication.

- **Email automation:**
  OutSystems provides tools for automating repetitive processes. In the AATLAS module, completed inspection emails are automatically parsed, validated, and recorded in the central database. This automation significantly reduced manual data entry and errors.

- **Workflow management:**
  The platform facilitates design and management of workflows. In the CORE module, inspection tasks are tracked in real-time, with status updates, notifications, and approval processes automated through workflow rules, ensuring smoother coordination among AIMD staff.

- **Web deployment:**
  OutSystems allows applications to be deployed as web-based platforms, which means AIMD staff can access VISTA through a browser without installing additional software. This ensures the platform is scalable, accessible across departments, and easy to maintain.

- **Reduction of fragmented platforms:**
  Previously, the AATLAS module was built using Power Apps and Power Automate, and inspection data were managed across multiple Excel files. OutSystems allowed consolidation of these separate tools into a single unified platform, reducing complexity, minimizing errors, and improving data integrity. Staff now have a single system for both completed and ongoing inspection tracking, eliminating the need to switch between multiple applications or manually review numerous Excel files.

Overall Impact on the Project:

Using OutSystems Service Studio made it possible to develop, test, and deploy modules rapidly, integrate multiple data sources, automate routine processes, and reduce reliance on fragmented tools such as Power Apps, Power Automate, and Excel. This approach also ensures that future modules or enhancements can be added efficiently and accessed seamlessly by AIMD staff.

3.3.2 Hardware Requirements

The project required hardware capable of supporting both development and deployment activities. The specifications include:

- **Desktop/Laptop** with minimum:
    - **Intel i5 processor** or equivalent
    - **8GB RAM**
    - **500GB HDD / SSD** for local storage
- **Stable internet connection** – Necessary for cloud access and server communication.
- **Server infrastructure** – For hosting the VISTA application and database, ensuring accessibility and performance for multiple users.

3.4 Results of Main Project

3.4.1 System Development

The VISTA project was developed using a structured methodology suitable for low-code development in OutSystems, ensuring that both modules were planned, designed, implemented, and tested systematically while allowing iterative improvements. The project focused on two main modules, the AATLAS Module, which automates completed inspection email processing, and the CORE Module, which tracks ongoing inspection activities and manages workflows.

Planning

During the planning phase, AIMD's existing inspection processes were reviewed to understand the current workflow and identify areas for improvement. Challenges such as multiple platforms for data entry, Excel-based tracking of inspection activities, and over thirty fragmented databases, forms, and dashboards were identified. Based on these findings, the internship scope was defined to include the completion of the AATLAS module and the continuation of CORE module development, with the specific database structures for these modules shown in **Figure 3.1**. Furthermore, project objectives, milestones, and deliverables were established to ensure alignment with AIMD's digitalization goals, following the proposed project schedule detailed in **Table 3.1**.

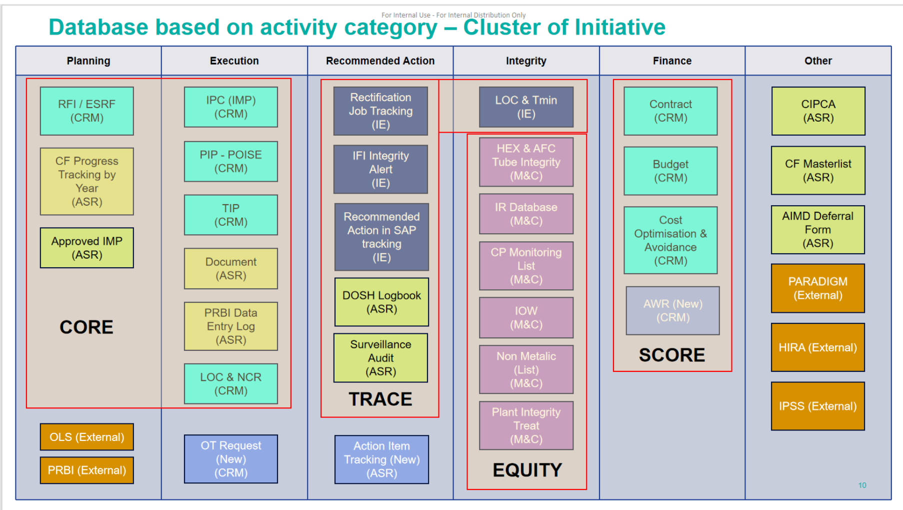


Figure 3.1 : Cluster of initiatives

Table 3.1 : Gantt Chart for Project Schedule

Task Name	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10
Planning										
Analysis										
Design & Implementation										
Testing										

<u>Analysis</u>

In the analysis phase, all relevant data sources, including existing databases, Excel files, and forms used by AIMD, were collected and reviewed to understand current inspection reporting and tracking practices. The inspection workflows for scheduling, completion reporting, and approval processes were analysed to identify inefficiencies, data duplication, and gaps in data visibility. This analysis provided a clear understanding of how information should be structured and managed within a centralized system.

A significant part of the analysis involved reviewing multiple Excel files that were previously used for inspection tracking and reporting. These Excel files were bootstrapped into OutSystems, allowing existing data to be migrated and structured within the platform. Through this process, the data contained in the Excel files was mapped to OutSystems entities, ensuring that historical records were preserved while enabling centralized data management moving forward.

Key data components required by the system were identified, such as inspection records, equipment and piping details, staff information, inspection status, reports, and approval data. Rather than manually designing data models, the analysis focused on defining entities and their attributes in OutSystems. Once these entities and attributes were defined, OutSystems automatically generated the underlying data models and relational structures, ensuring consistency, reducing manual effort, and minimizing potential design errors.

Functional requirements were identified during this phase, including automated handling of completed inspection emails, real-time tracking of ongoing inspection activities, workflow approvals, and centralized reporting. Non-functional requirements, such as system reliability, data accuracy, usability, and web-based accessibility, were also considered to ensure the system supports daily operational needs effectively.

Overall, the analysis phase emphasized understanding AIMD's business processes and existing data sources, while leveraging OutSystems' built-in capabilities to manage relational data models and integrate legacy Excel data into a unified digital platform. The culmination of this evaluation and the structural requirements of the system are represented in the final analysis shown in **Figure 3.2**.

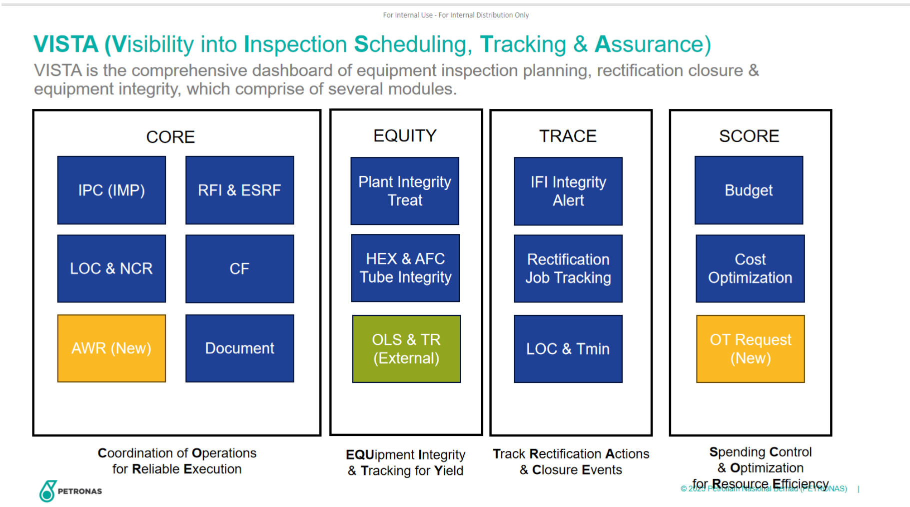


Figure 3.2 *: Summary of analysis findings for VISTA modules*

Design and Implementation

During the design and implementation phase, the AATLAS module was developed to automatically process completed inspection emails and update the VISTA database. This reduced manual data entry and errors caused by reliance on multiple Excel files and previous tools such as Power Apps and Power Automate. The CORE module was designed to track ongoing inspection tasks in real-time, integrating status updates, workflow approvals, and notifications. Data structures were created according to the ERD, while user interfaces were built using OutSystems screens and widgets, optimized for usability. Automation logic for email processing, workflow triggers, and report generation was implemented, and both modules were integrated into a single platform, reducing reliance on multiple tools and improving data integrity.

Testing

Testing was conducted at multiple levels to ensure the system met requirements and functioned correctly. Unit testing validated individual features, including email parsing, task updates, and dashboards. Integration testing confirmed proper communication between AATLAS and CORE modules and correct updates in the central database. Functional testing verified the system against requirements, ensuring accurate inspection updates, workflow notifications, and consolidated reporting. User acceptance testing (UAT) was performed with AIMD staff to confirm that the system was practical, efficient, and user-friendly.

Results

The project successfully achieved its intended outcomes. The AATLAS module automates completed inspection email processing and updates records in the central database, eliminating the need for multiple Excel files and previous Power Apps solutions. The CORE module tracks ongoing inspections in real-time, manages workflows, and provides automated notifications to staff. By consolidating fragmented databases, forms, and dashboards, the VISTA platform provides a centralized system that reduces manual work, minimizes errors, and improves inspection tracking and reporting efficiency. Additionally, dashboards and summary reports enable management to monitor inspection progress and make data-driven decisions.

Overall, the VISTA project demonstrates how the AATLAS and CORE modules, integrated within a single low-code platform, can streamline inspection processes, reduce manual effort, and provide a unified, transparent view of all inspection activities for AIMD staff and management.

3.4.2 Main Project Interface

Figures 3.3 – 3.7 show a sample interface of the AATLAS module:

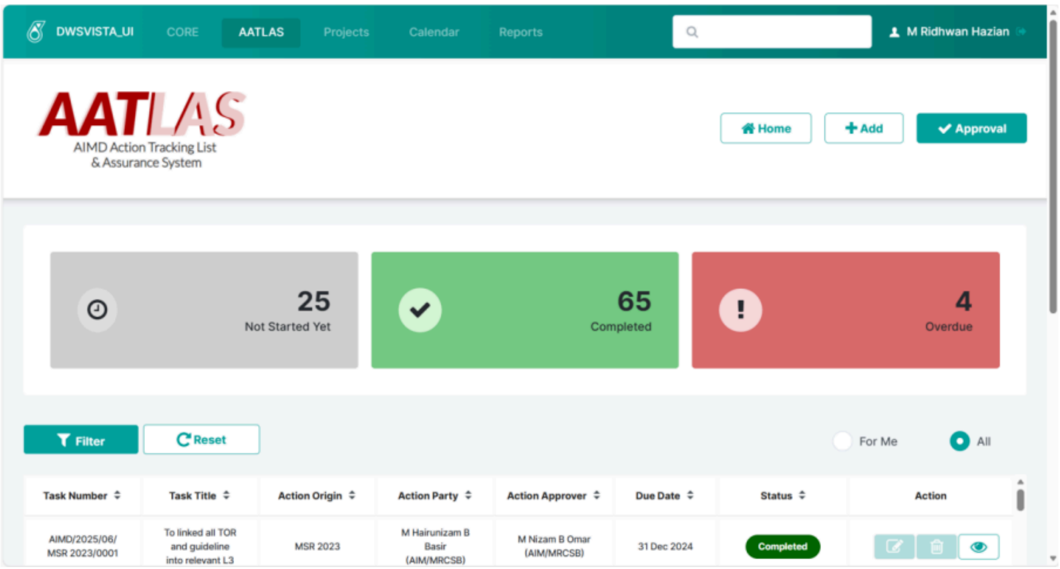
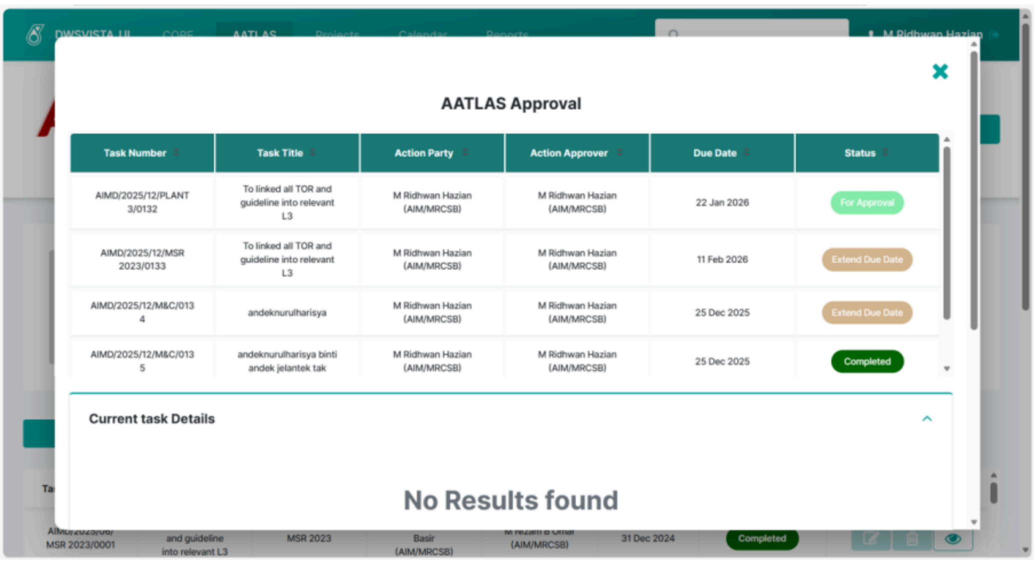
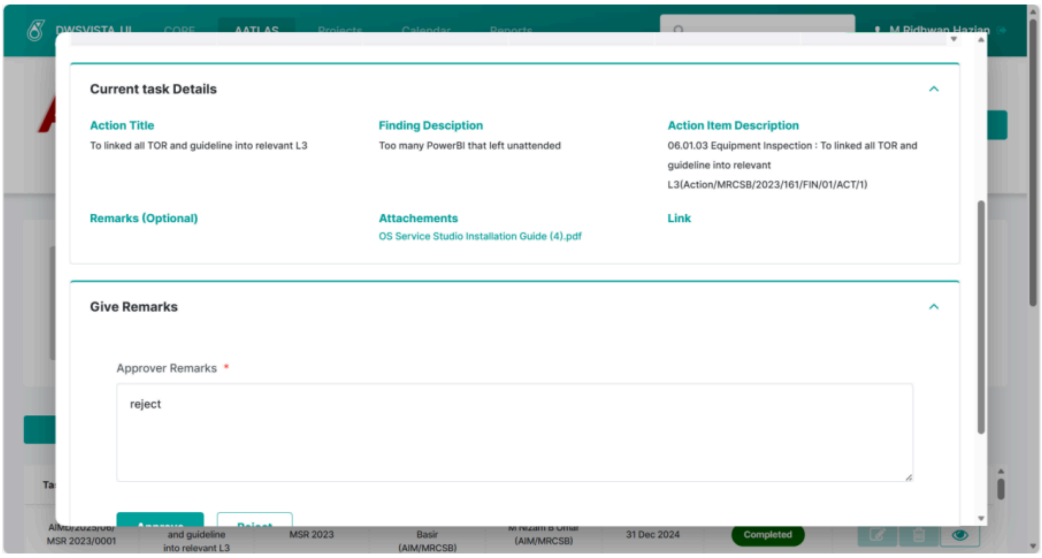


Figure 3.3 *: Shows AATLAS main page*





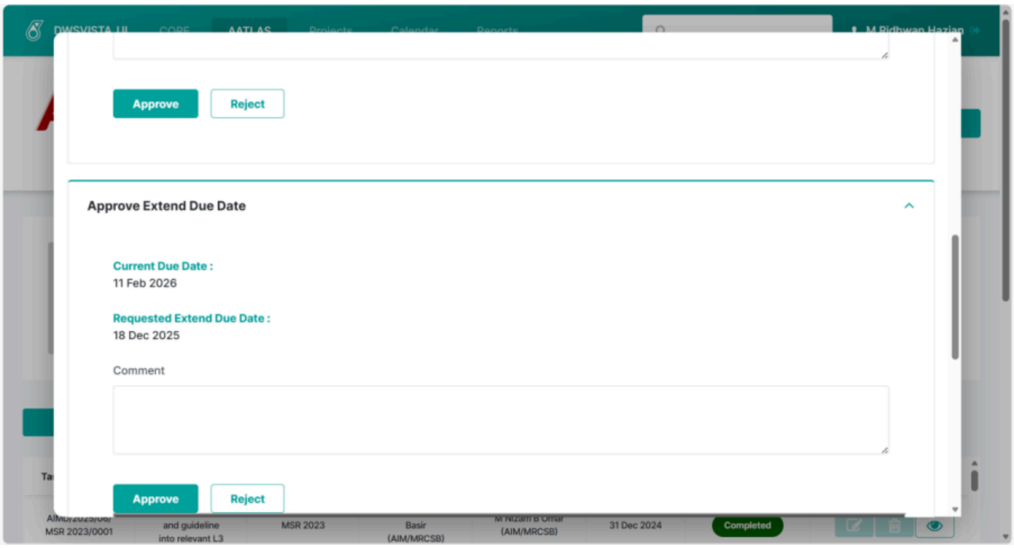
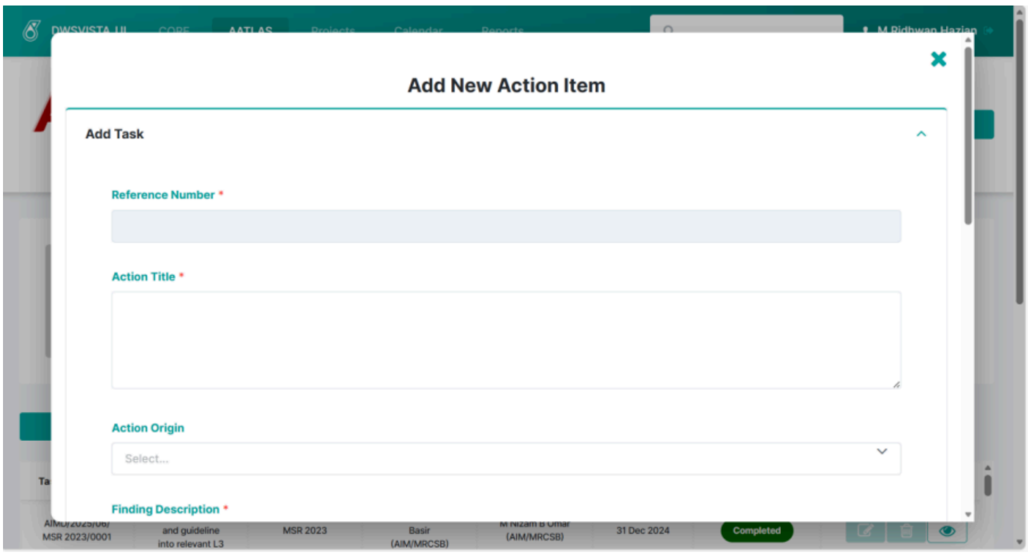
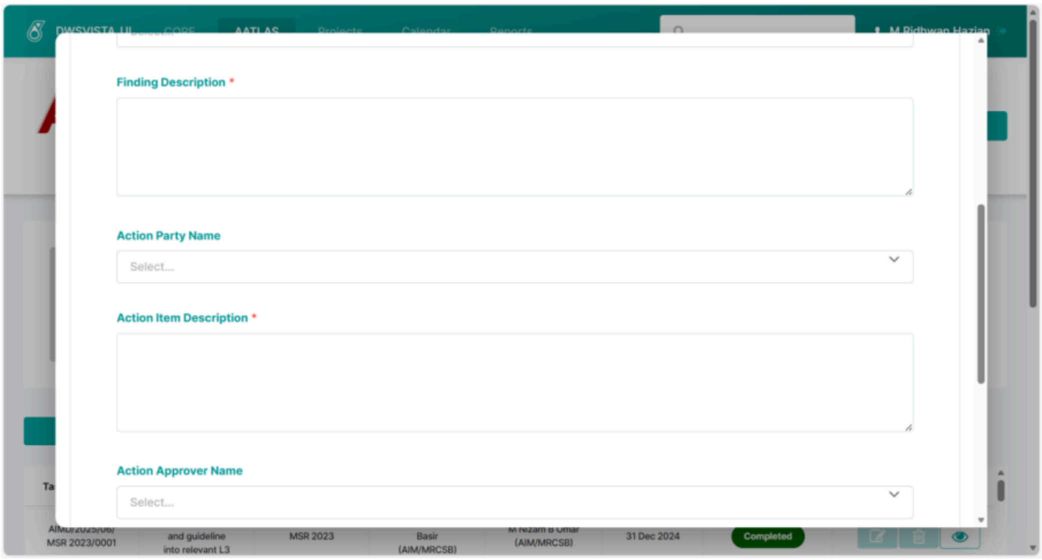
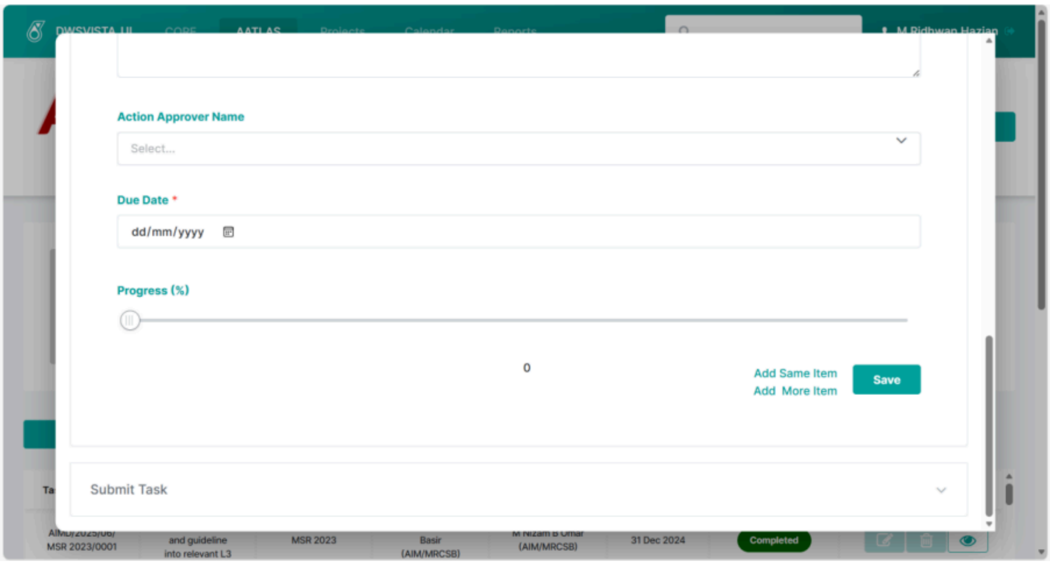


Figure 3.4 *: Shows AATLAS Approval Pop-Up*







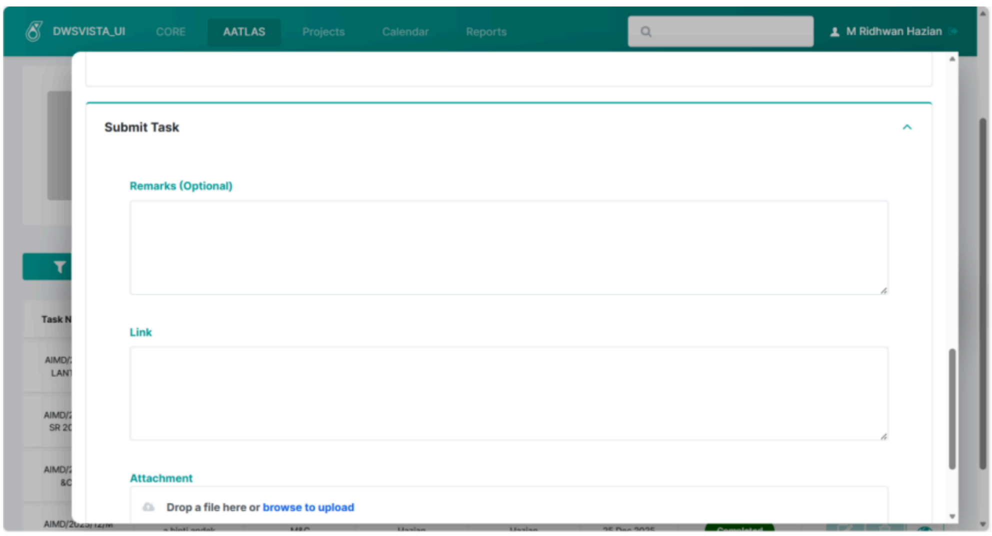


***Figure 3.5** : Shows AATLAS Edit/Add Pop-Up*

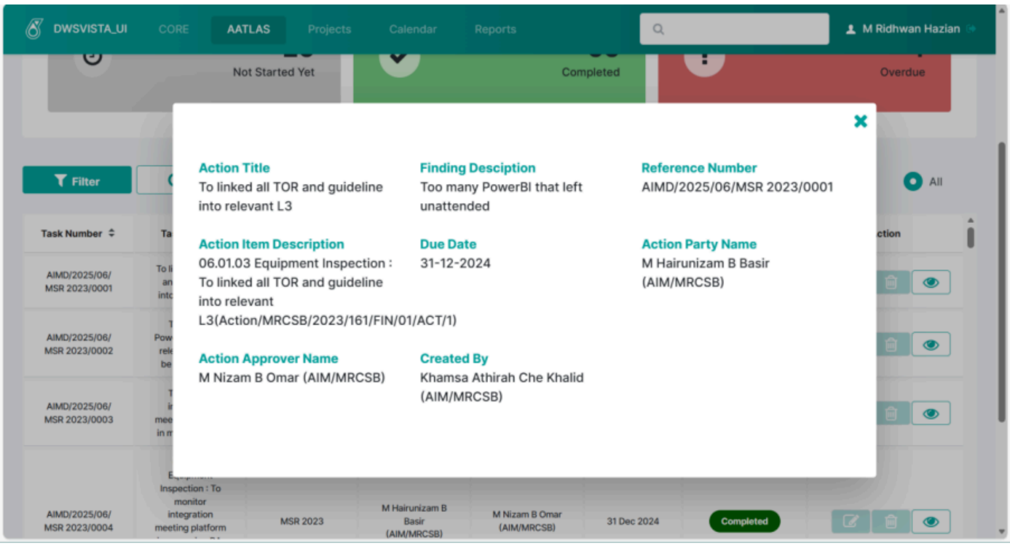


Figure 3.6 : *Shows AATLAS view button on click Pop-Up*

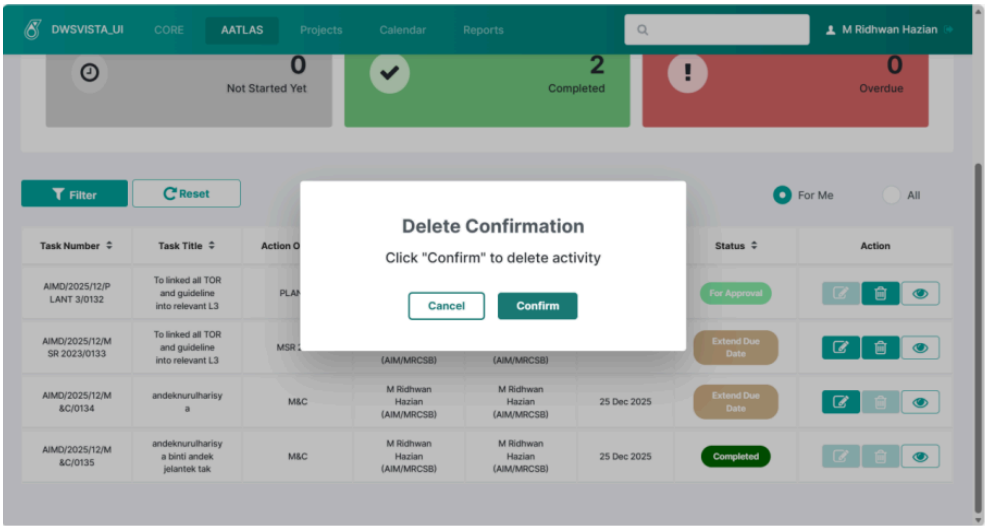
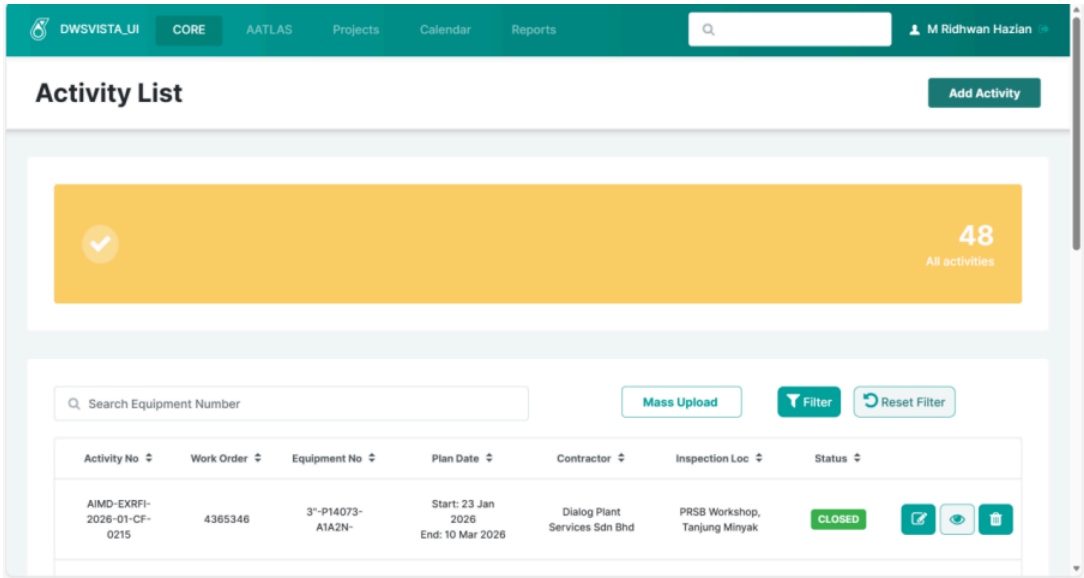


Figure 3.7 : *Shows delete button on click Pop-Up*

Figures 3.8 – 3.11 show a sample interface of the CORE module :



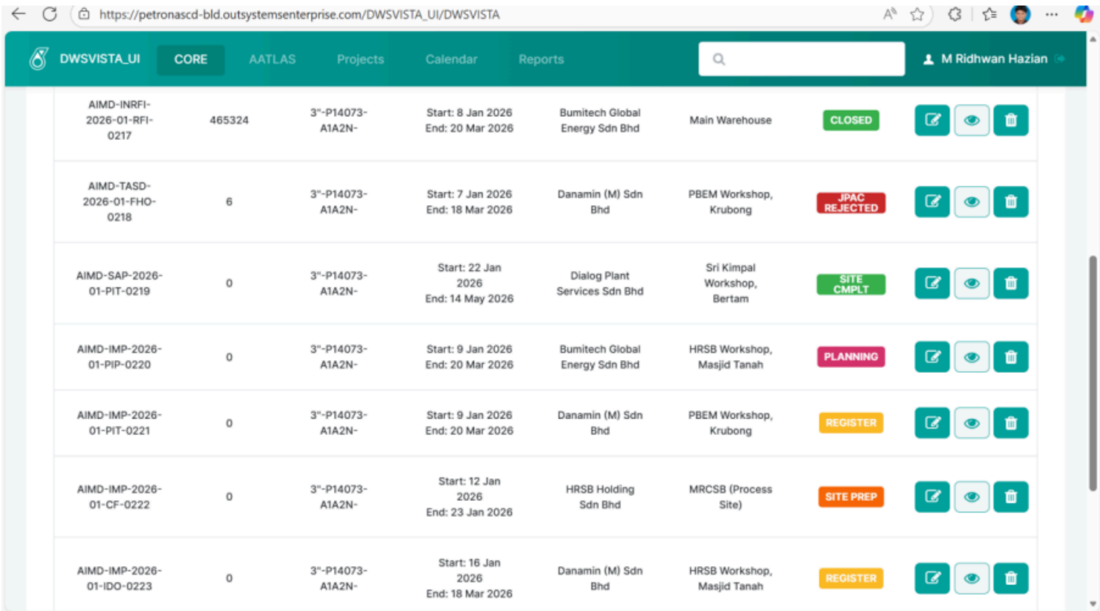


Figure 3.8 *: Shows CORE main page*

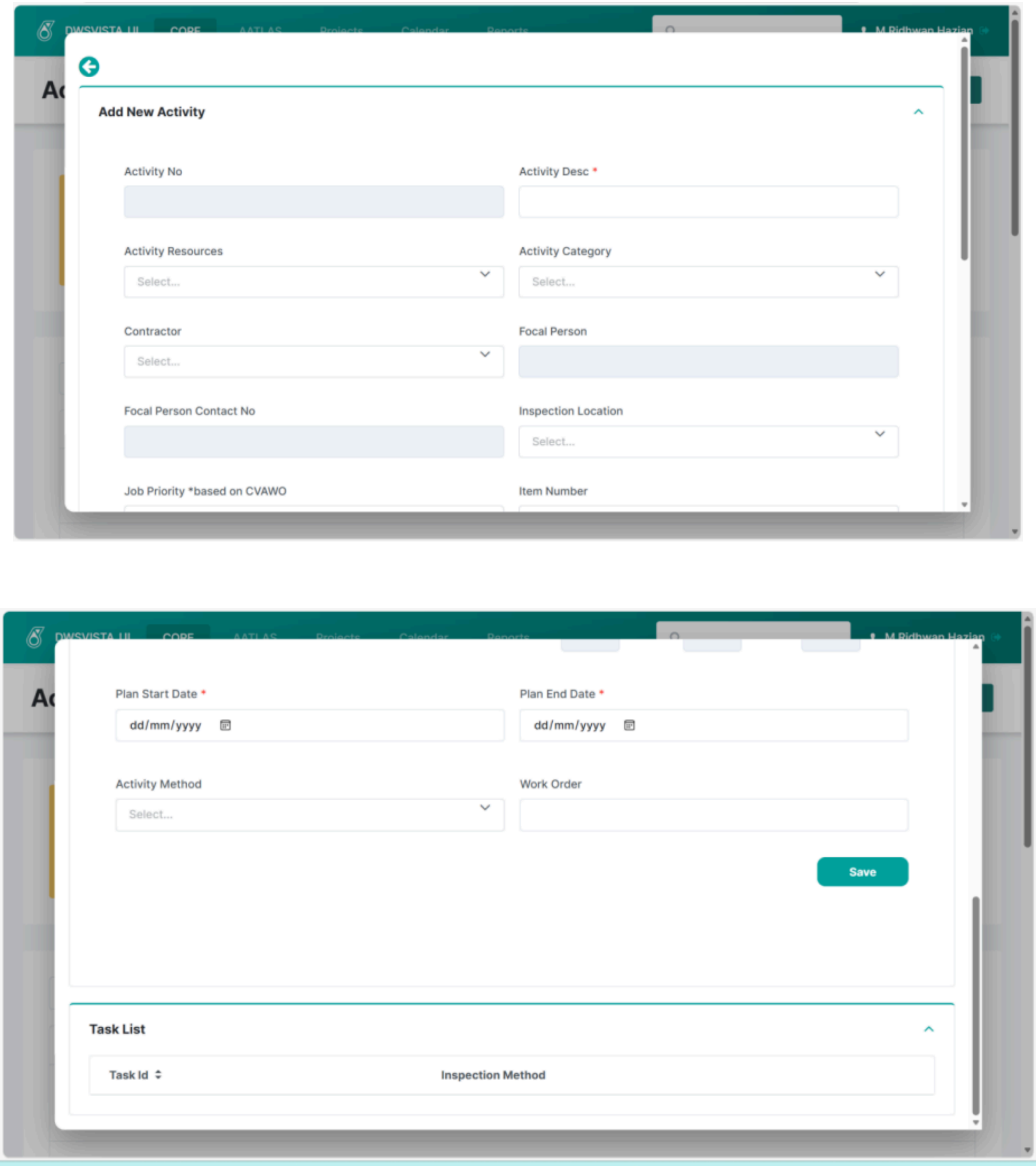
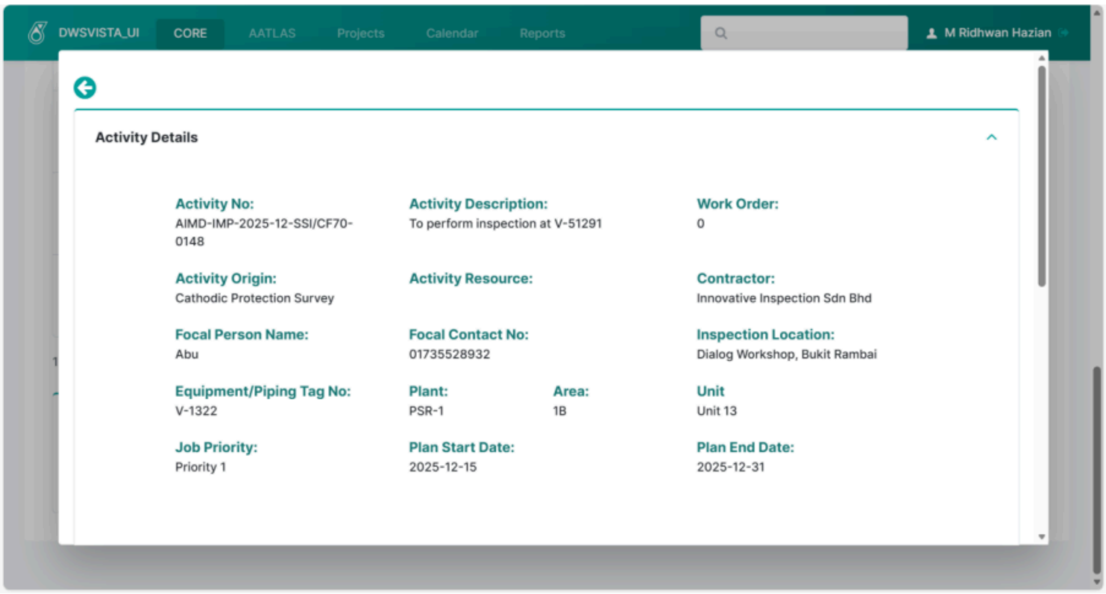


Figure 3.9 *: Shows CORE Add activity Pop-Up*



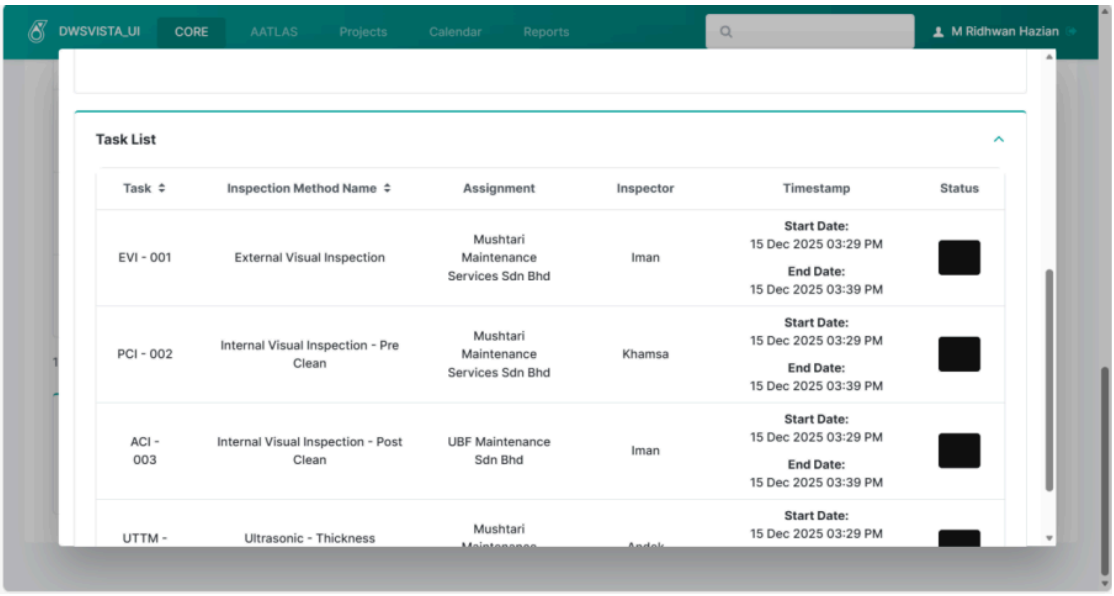
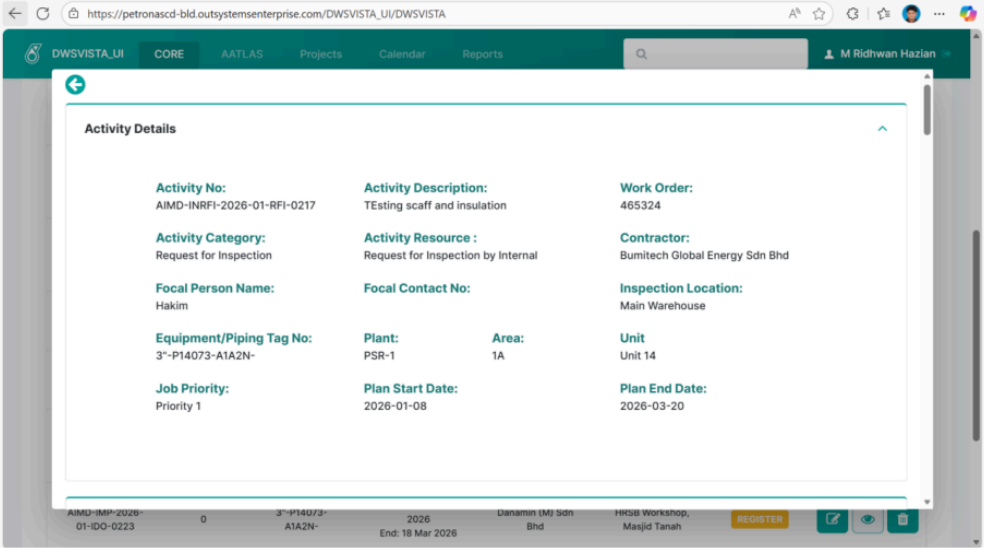
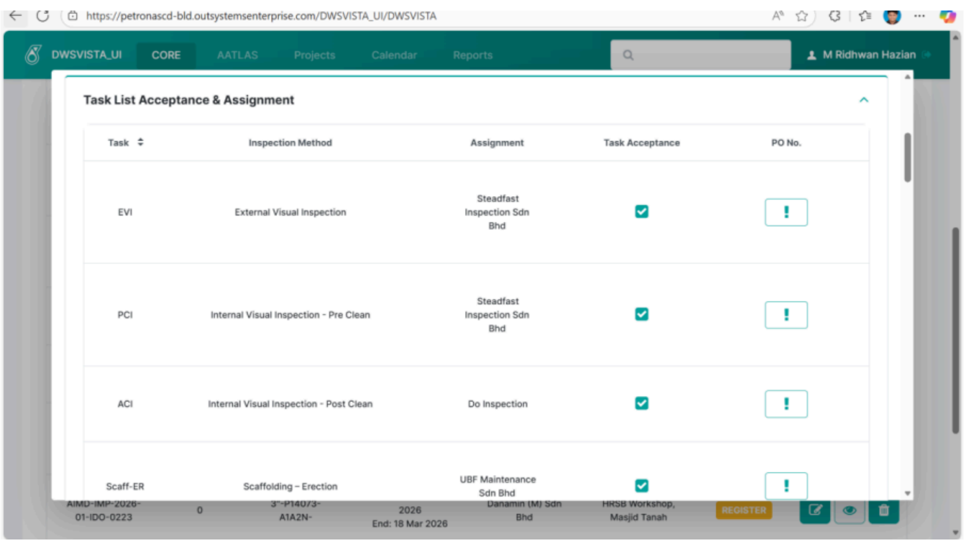
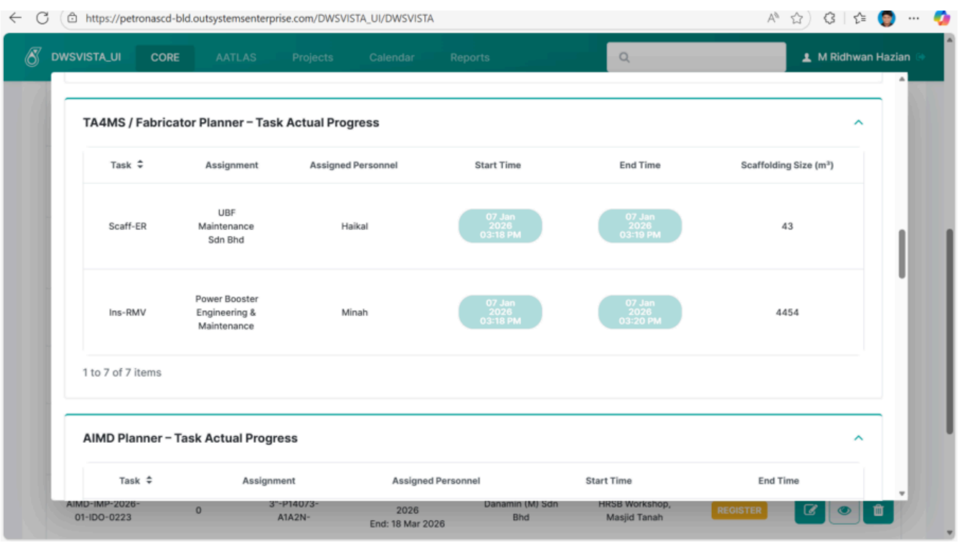
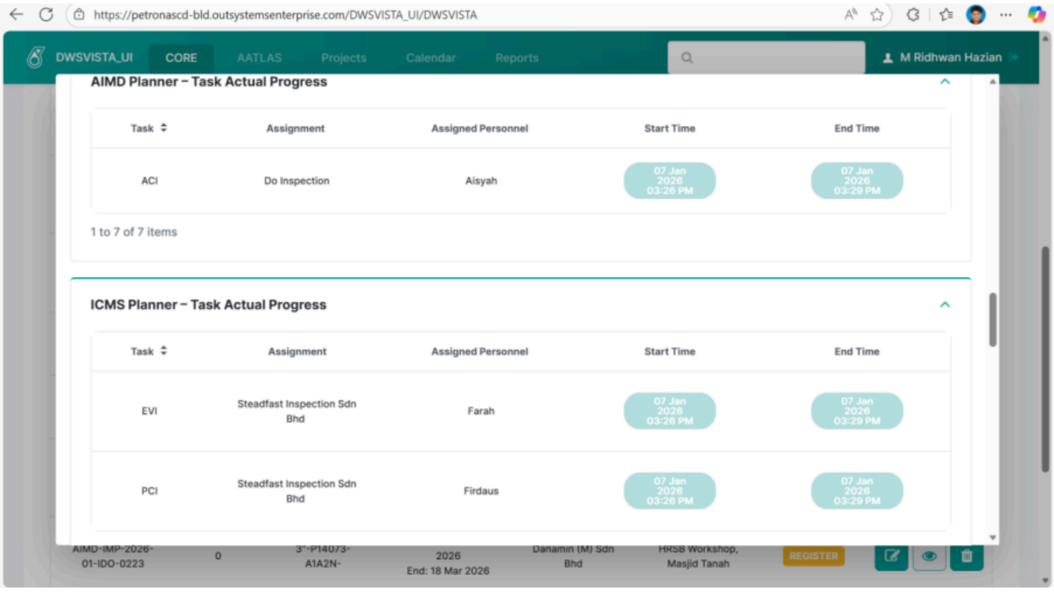


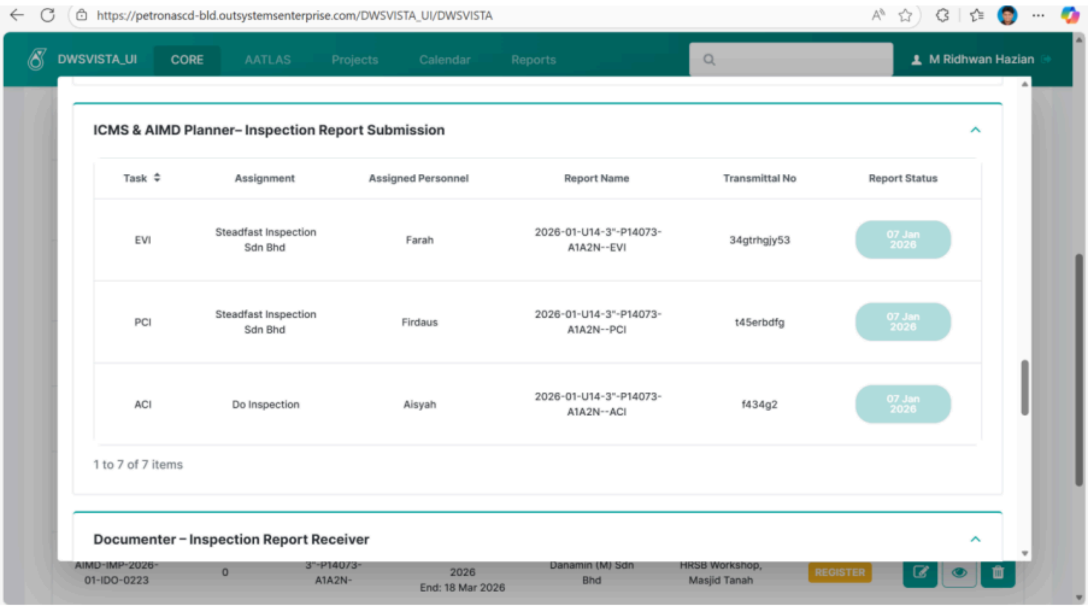
***Figure 3.10** : CORE view Pop-Up on click*

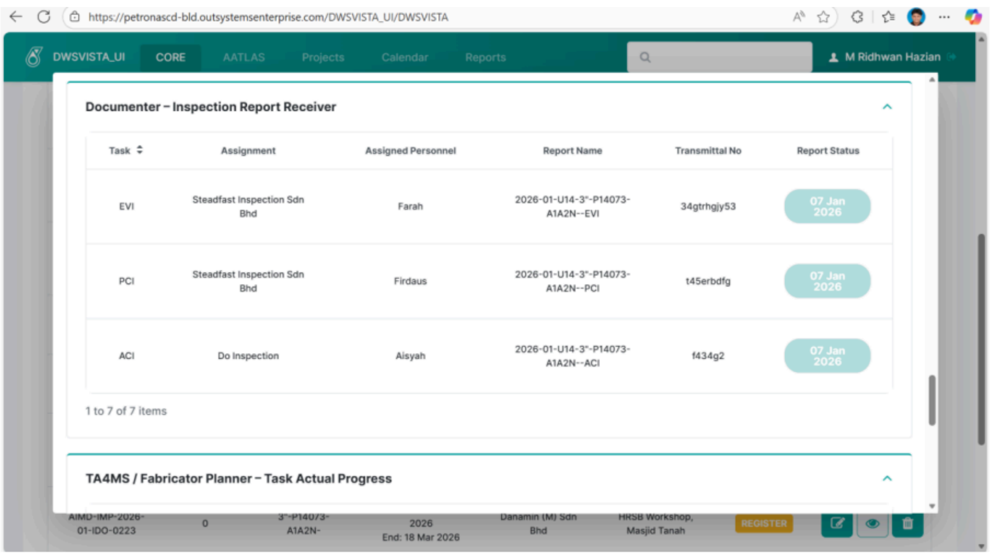

https://petronascd-bld.outsystemsenterprise.com/DWSVISTA_UI/DWSVISTA
DWSVISTA_UI
CORE
AATLAS
Projects
Calendar
Reports
M Ridhwan Hazian
Activity Details
Activity No:
AIMD-INRFI-2026-01-RFI-0217
Activity Description:
TEsting scaff and insulation
Work Order:
465324
Activity Category:
Request for Inspection
Activity Resource :
Request for Inspection by Internal
Contractor:
Bumitech Global Energy Sdn Bhd
Focal Person Name:
Hakim
Focal Contact No:
Inspection Location:
Main Warehouse
Equipment/Piping Tag No:
3"-P14073-A1A2N-
Plant:
PSR-1
Area:
1A
Unit
Unit 14
Job Priority:
Priority 1
Plan Start Date:
2026-01-08
Plan End Date:
2026-03-20

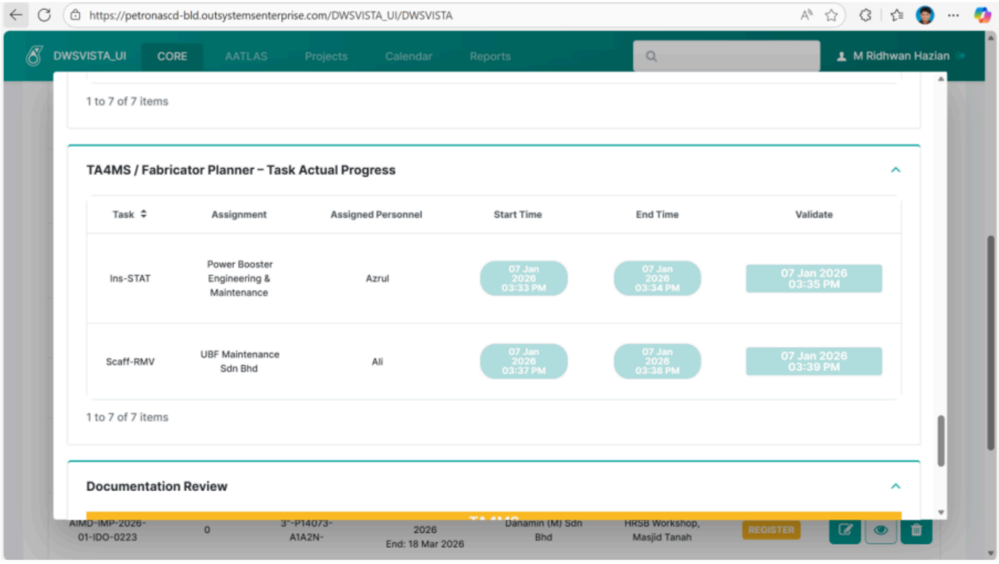

https://petronascd-bld.outsystemsenterprise.com/DWSVISTA_UI/DWSVISTA
DWSVISTA_UI
CORE
AATLAS
Projects
Calendar
Reports
M Ridhwan Hazian
Task List Acceptance & Assignment
Task
Inspection Method
Assignment
Task Acceptance
PO No.
EVI
External Visual Inspection
Steadfast Inspection Sdn Bhd
PCI
Internal Visual Inspection - Pre Clean
Steadfast Inspection Sdn Bhd
ACI
Internal Visual Inspection - Post Clean
Do Inspection
Scaff-ER
Scaffolding - Erection
UBF Maintenance Sdn Bhd


https://petronascd-bld.outsystemsenterprise.com/DWSVISTA_UI/DWSVISTA
DWSVISTA_UI
CORE
AATLAS
Projects
Calendar
Reports
M Ridhwan Hazian
TA4MS / Fabricator Planner – Task Actual Progress
Task
Assignment
Assigned Personnel
Start Time
End Time
Scaffolding Size (m³)
Scaff-ER
UBF Maintenance Sdn Bhd
Haikal
07 Jan 2026 03:18 PM
07 Jan 2026 03:19 PM
43
Ins-RMV
Power Booster Engineering & Maintenance
Minah
07 Jan 2026 03:18 PM
07 Jan 2026 03:20 PM
4454
1 to 7 of 7 items
AIMD Planner – Task Actual Progress
Task
Assignment
Assigned Personnel
Start Time
End Time


https://petronascd-bld.outsystemsenterprise.com/DWSVISTA_UI/DWSVISTA
DWSVISTA_UI
CORE
AATLAS
Projects
Calendar
Reports
M Ridhwan Hazian
AIMD Planner – Task Actual Progress
Task
Assignment
Assigned Personnel
Start Time
End Time
ACI
Do Inspection
Aisyah
07 Jan 2026 03:26 PM
07 Jan 2026 03:29 PM
1 to 7 of 7 items
ICMS Planner – Task Actual Progress
Task
Assignment
Assigned Personnel
Start Time
End Time
EVI
Steadfast Inspection Sdn Bhd
Farah
07 Jan 2026 03:26 PM
07 Jan 2026 03:29 PM
PCI
Steadfast Inspection Sdn Bhd
Firdaus
07 Jan 2026 03:26 PM
07 Jan 2026 03:29 PM
AIMD-IMP-2026-01-IDO-0223
0
3"-P14073-A1A2N-
2026 End: 18 Mar 2026
Danamin (M) Sdn Bhd
HRSB Workshop, Masjid Tanah
REGISTER


https://petronascd-bld.outsystemsenterprise.com/DWSVISTA_UI/DWSVISTA
DWSVISTA_UI
CORE
AATLAS
Projects
Calendar
Reports
M Ridhwan Hazian
ICMS & AIMD Planner– Inspection Report Submission
Task
Assignment
Assigned Personnel
Report Name
Transmittal No
Report Status
EVI
Steadfast Inspection Sdn Bhd
Farah
2026-01-U14-3"-P14073-A1A2N--EVI
34gtrhgjy53
07 Jan 2026
PCI
Steadfast Inspection Sdn Bhd
Firdaus
2026-01-U14-3"-P14073-A1A2N--PCI
t45erbdfg
07 Jan 2026
ACI
Do Inspection
Aisyah
2026-01-U14-3"-P14073-A1A2N--ACI
f434g2
07 Jan 2026
1 to 7 of 7 items
Documenter – Inspection Report Receiver
AIMD-IMP-2026-01-IDO-0223
0
3"-P14073-A1A2N-
2026 End: 18 Mar 2026
Danamin (M) Sdn Bhd
HRSB Workshop, Masjid Tanah
REGISTER


https://petronascd-bld.outsystemsenterprise.com/DWSVISTA_UI/DWSVISTA
DWSVISTA_UI
CORE
AATLAS
Projects
Calendar
Reports
M Ridhwan Hazian
Documenter – Inspection Report Receiver
Task
Assignment
Assigned Personnel
Report Name
Transmittal No
Report Status
EVI
Steadfast Inspection Sdn Bhd
Farah
2026-01-U14-3"-P14073-A1A2N--EVI
34gtrhgjy53
07 Jan 2026
PCI
Steadfast Inspection Sdn Bhd
Firdaus
2026-01-U14-3"-P14073-A1A2N--PCI
t45erbdfg
07 Jan 2026
ACI
Do Inspection
Aisyah
2026-01-U14-3"-P14073-A1A2N--ACI
f434g2
07 Jan 2026
1 to 7 of 7 items
TA4MS / Fabricator Planner – Task Actual Progress
AIMD-IMP-2026-01-IDO-0223
0
3"-P14073-A1A2N-
2026 End: 18 Mar 2026
Danamin (M) Sdn Bhd
HRSB Workshop, Masjid Tanah
REGISTER



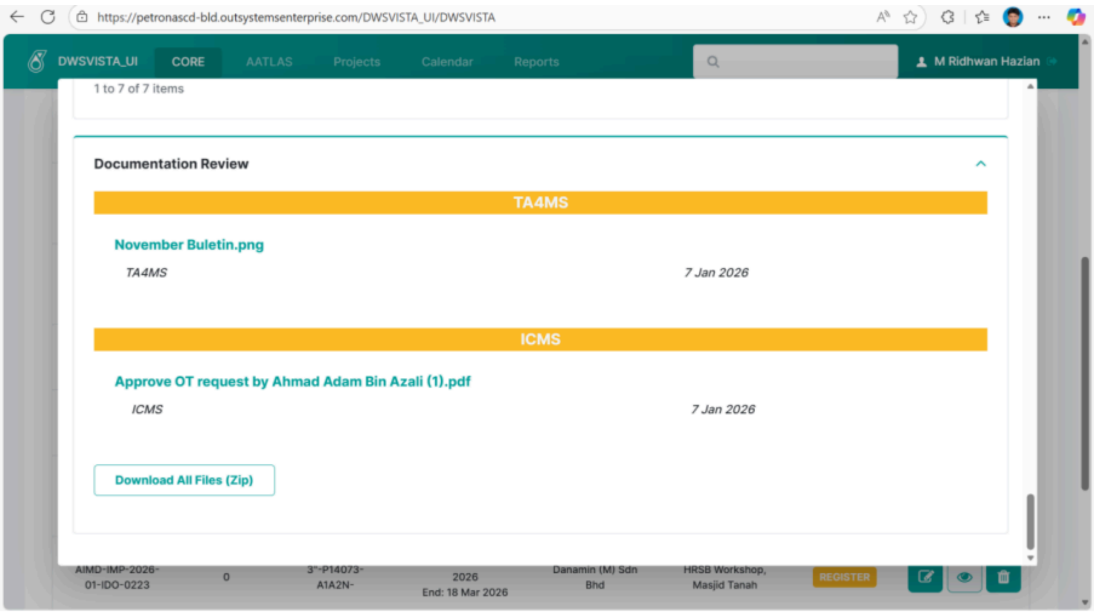


__Figure 3.11__ : Shows CORE edit on click Pop-Up

3.4.3 Use Case

<u>AATLAS</u>

Scenario 1 : Editing AATLAS Action Item in **Figure 3.12 - 3.14**.

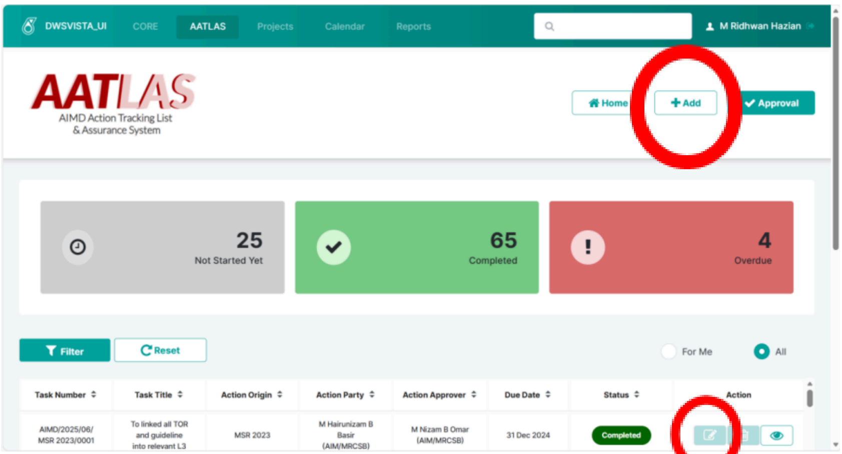


Figure 3.12 *: Shows Button Edit(down) and Add(up) on AATLAS main page*

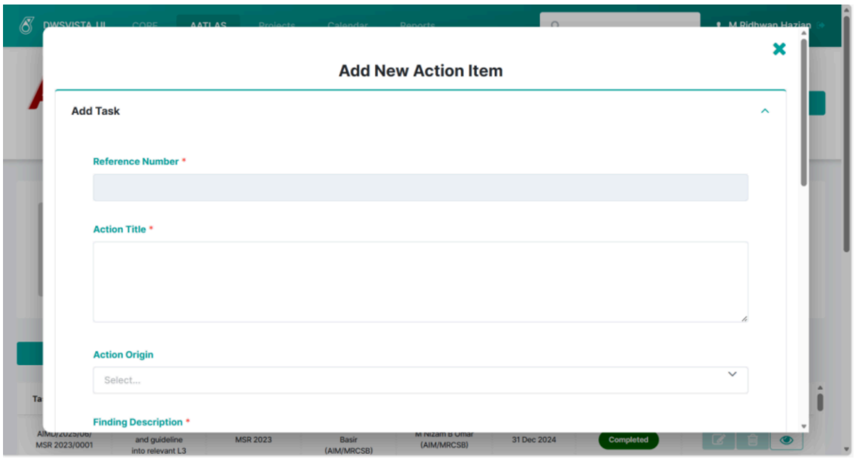


Figure 3.13 *: Shows when AATLAS button Edit is clicked a Pop-Up appear*

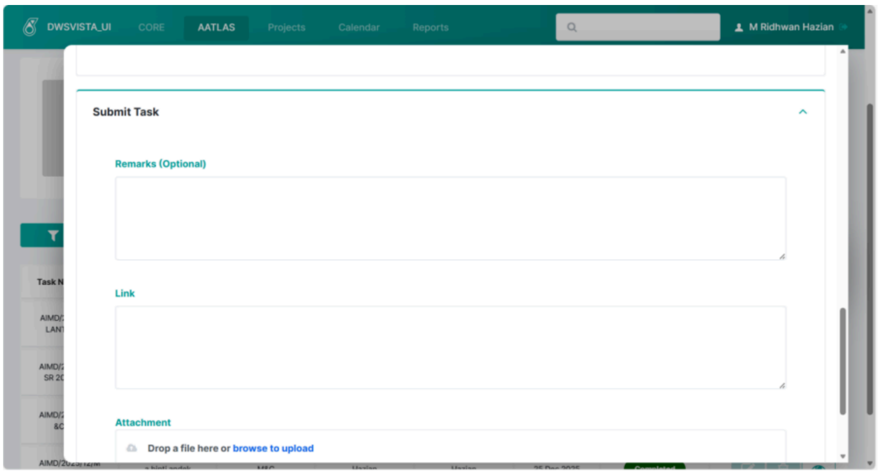


Figure 3.14 *: Shows additional information on Edit button click*

In the AATLAS module, action parties manage tasks through a shared popup screen designed for both adding and editing items. Clicking the Add button triggers a popup titled "Add New Action Item," which provides empty fields for new details such as the action title, party, and approver, with the resulting interface displayed in **Figure 3.12**,

Figure 3.13, and **Figure 3.14**. Conversely, clicking the Edit button for an existing item causes the title to dynamically change to "Edit Action Item," with the output being a form pre-filled with existing data. To maintain security, the system validates the current user and automatically disables the button for unauthorized personnel. This approach ensures a consistent user experience while strictly protecting data integrity by restricting modifications to the assigned party.

Scenario 2 : Deleting an AATLAS Action Item on **Figure 3.15 and 3.16**.

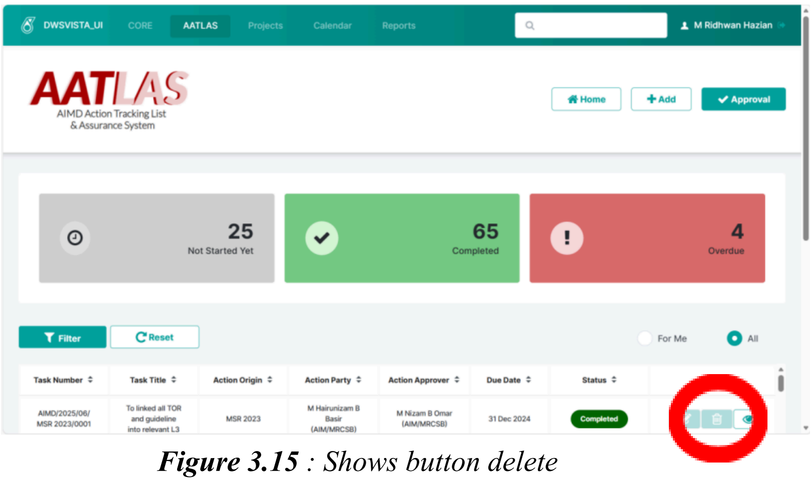


Figure 3.15 : *Shows button delete*

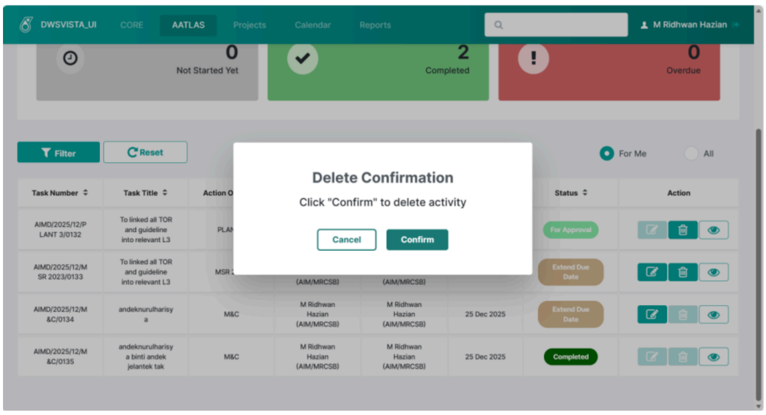


Figure 3.16 : *Shows when button delete is clicked then a Pop-Up appear*

In the AATLAS module, users can remove an action item via the Delete button on the main page, which corresponds to the layout in **Figure 3.15**. Clicking this button triggers a confirmation popup to prevent accidental data loss, resulting in the prompt seen in **Figure 3.16**. This popup offers two distinct options: Cancel or Confirm. Selecting Cancel aborts the request, leaving the action item unchanged. Conversely, selecting

Confirm executes the permanent removal of the item from the database. This confirmation mechanism serves as a critical safeguard, ensuring that all deletions are intentional and maintaining the overall data accuracy of the system.

Scenario 3 : Viewing an AATLAS Action Item on **Figure 3.17 and 3.18.**

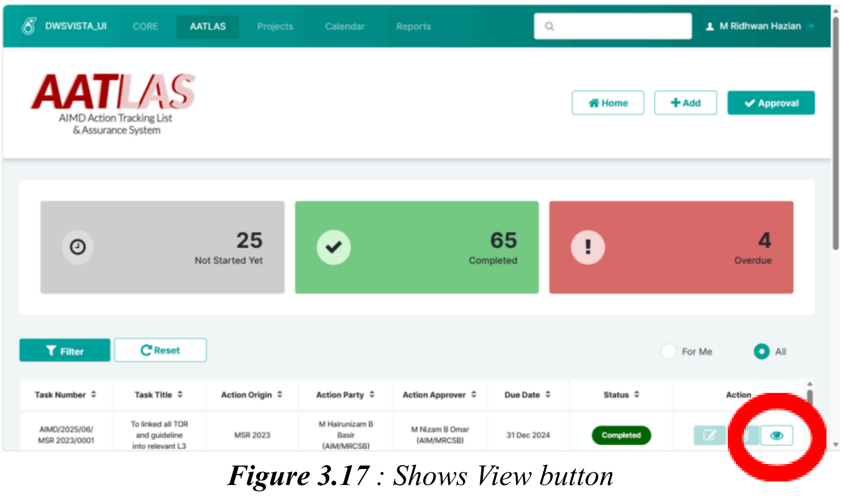


Figure 3.17 : Shows View button

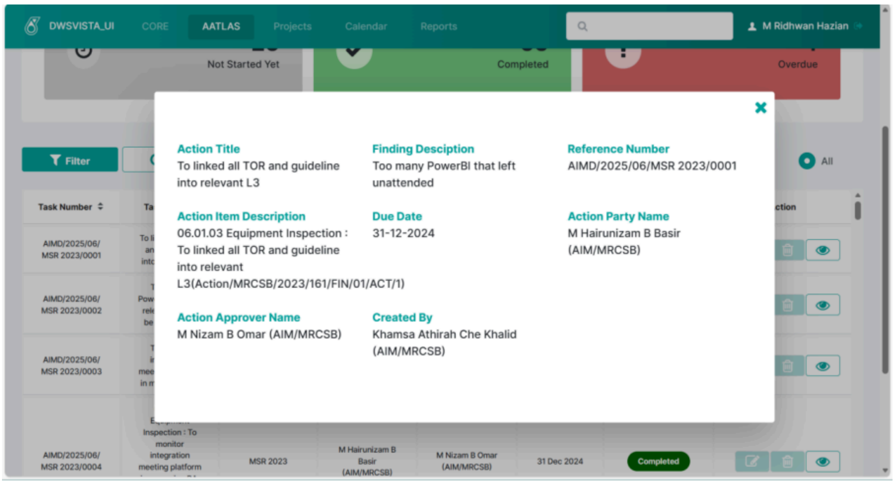


***Figure 3.18** : Shows View button on click a Pop-Up will appear*

In the AATLAS module, users can access the full details of an action item via the View button on the main page, which corresponds to the interface layout in **Figure 3.17**. Clicking this button triggers a popup window that retrieves and displays all relevant information for the selected item, resulting in the view seen in **Figure 3.18**. This detail pane presents the action title, party, approver, due date, status, and any additional remarks. To safeguard the information, the system renders this popup in a read-only mode, which effectively prevents accidental modifications. This function ensures users can quickly review tasks with full transparency while maintaining the integrity of the underlying data.

Scenario 4 : Approving or Rejecting an Action Item on **Figure 3.19 and 3.20**.

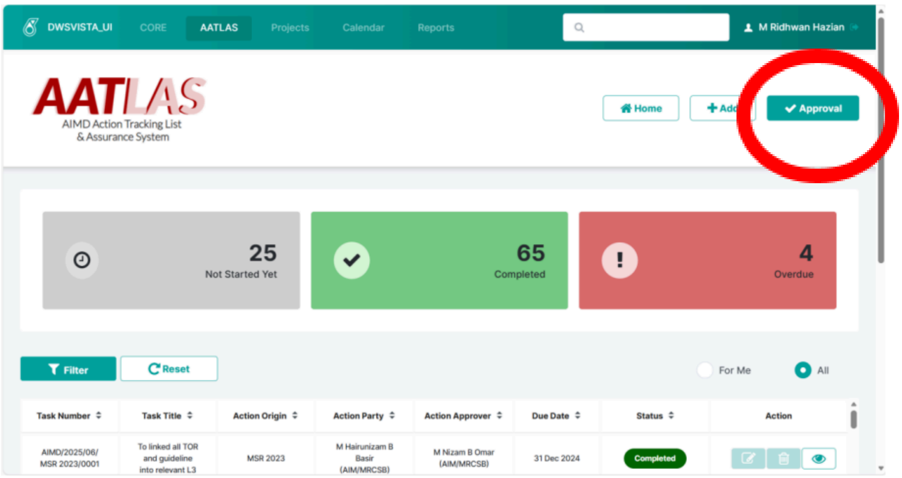
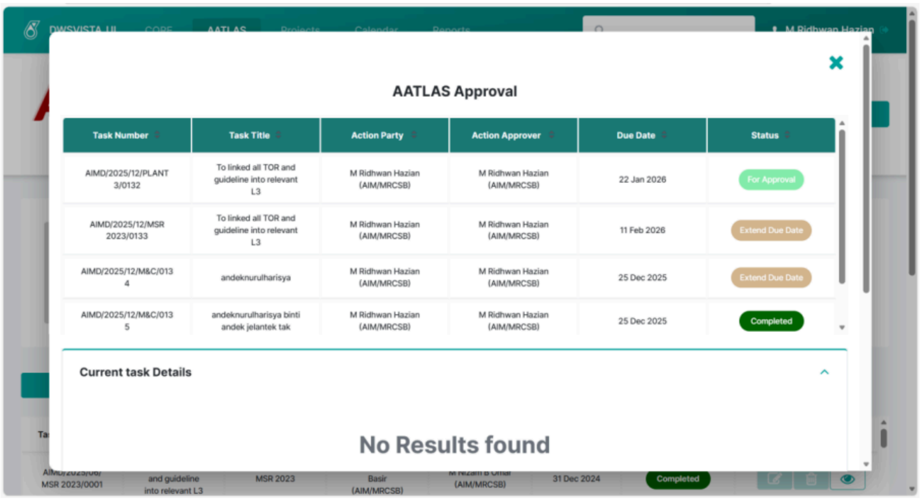
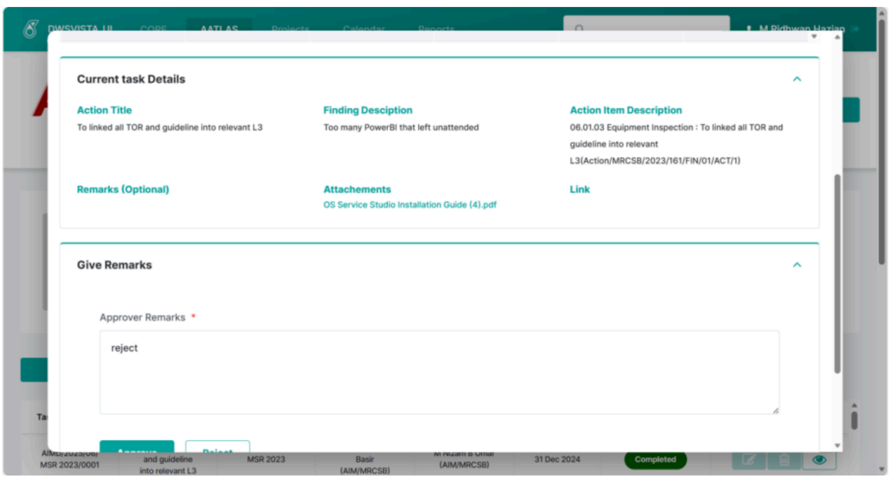


Figure 3.19 *: Shows button Approval*





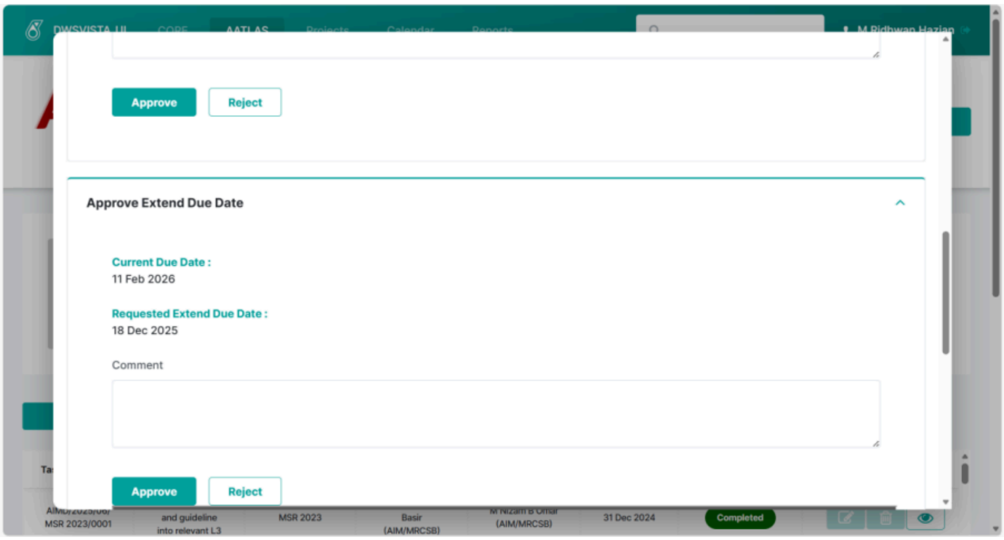


***Figure 3.20** : Shows Approval button on click a Pop-Up appear*

In the AATLAS module, approvers manage assigned items through a dedicated Approval section. Selecting the Approval button triggers a popup window, which corresponds to the interface layout in **Figure 3.19**. To maintain a focused workflow, the system filters the table to display only items with the status "For Approval," "Completed," or "Extend Due Date". When a row is selected, the system retrieves the full item details and activates a mandatory remarks field; the outcome is that a decision cannot be finalized until a comment is entered. For standard submissions, the approver may either approve or reject the item, which immediately updates the status in the database. In cases involving an "Extend Due Date" request, the interface dynamically updates to present additional extension details for scrutiny. This rigorous process, with the resulting decision states captured in **Figure 3.20**, ensures that all approvals are properly documented and controlled.

Scenario 5 : Add Activity Button On click CORE on **Figure 3.21 and 3.22**.

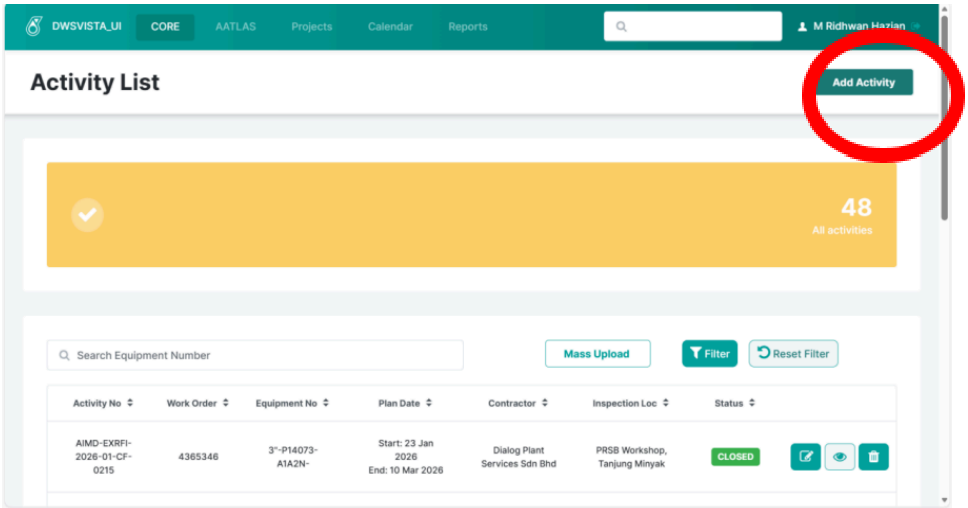


Figure 3.21 *: Shows button Add Activity*

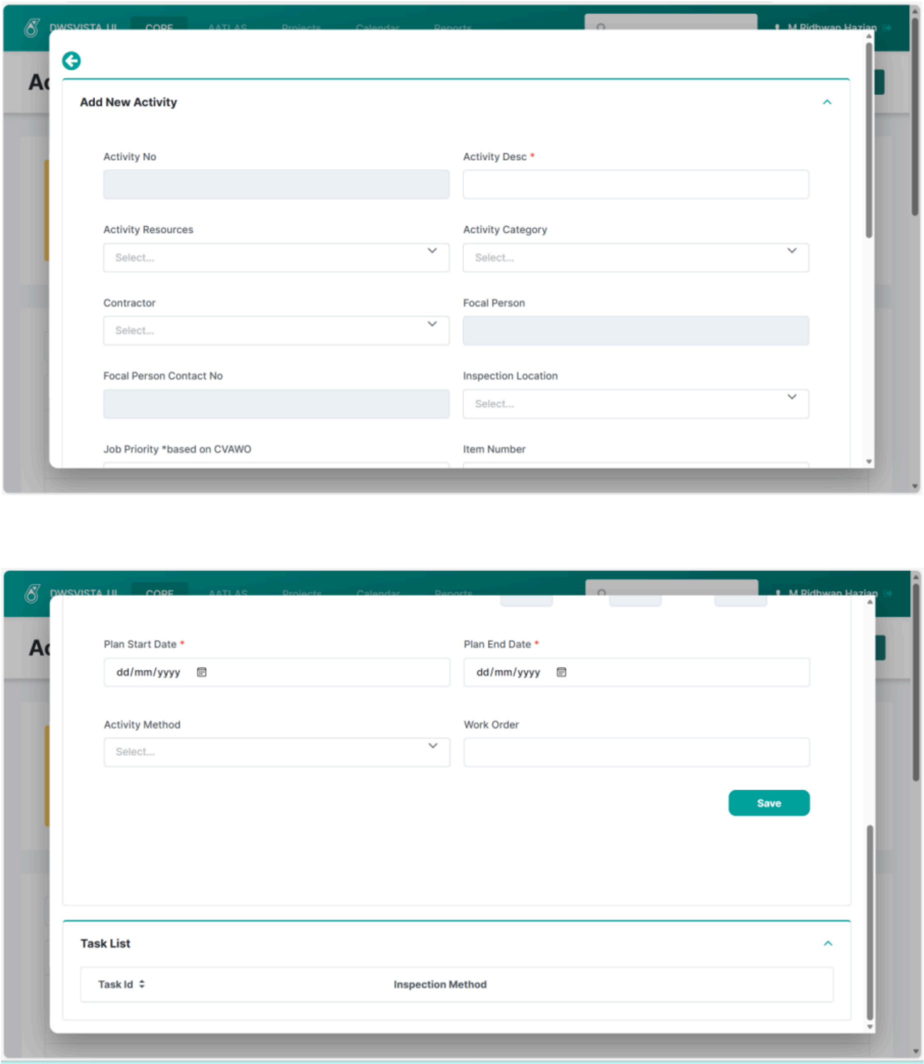
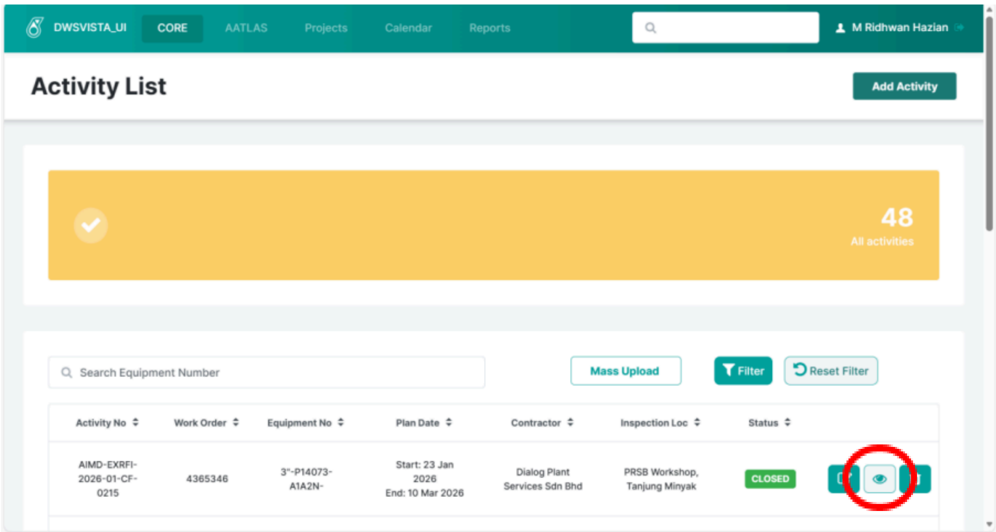
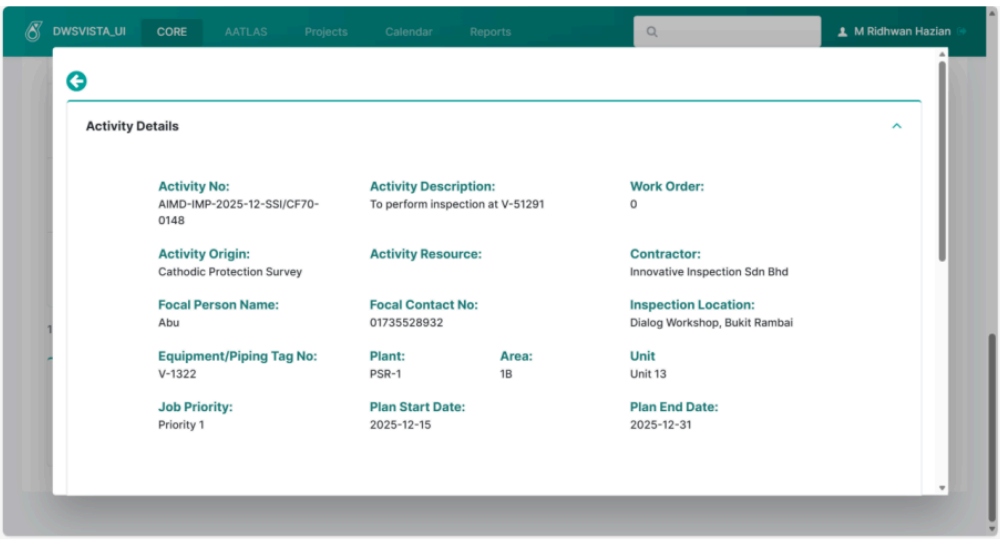


Figure 3.22 *: Shows Add Button on click shows Pop-Up*

In the CORE module, users initiate a new inspection activity via the Add Activity button on the main page, an action that invokes the interface shown in **Figure 3.21**. This selection triggers a popup window, yielding the data entry form seen in **Figure 3.22**, where all relevant inspection details must be provided. These fields are designed to capture critical data such as the inspection type, equipment or piping references, the responsible company, and the assigned contractor. Once the required information is populated, the system processes the form submission to officially register the activity. This structured function ensures that all inspection data is recorded consistently, which enables accurate tracking and seamless coordination between internal AIMD teams and external contractors.

Scenario 6 : Viewing an Inspection Activity in CORE on **Figure 3.23**.





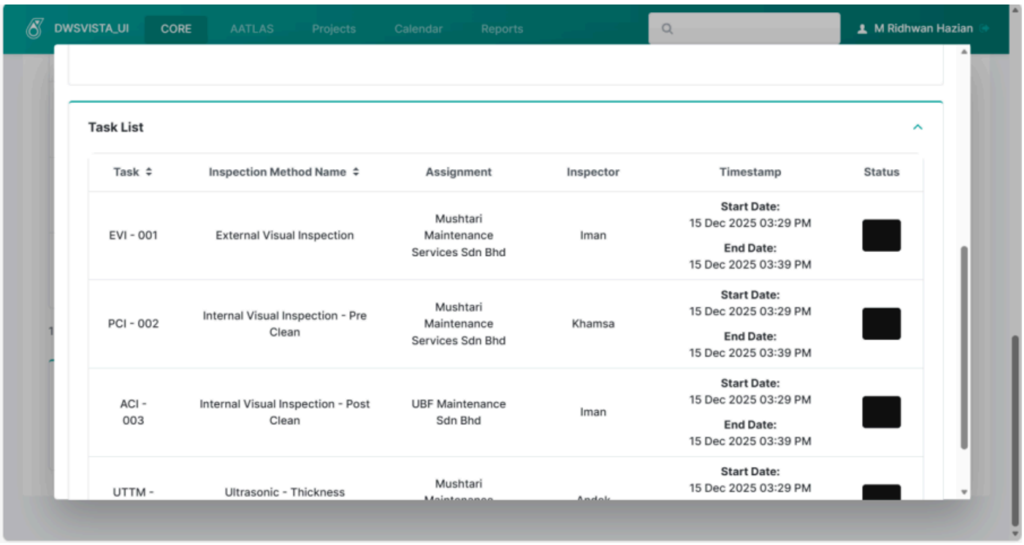


Figure 3.23 *: Shows view activity button on click Pop-Up*

In the CORE module, users can access the full details of any inspection activity by clicking the View button within the activity list. This action brings up the popup window shown in **Figure 3.23**, which consolidates all relevant information—including inspection data, assigned contractors, and the specific list of associated tasks. By clearly indicating the status of each task, the system allows users to monitor real-time progress across all involved parties. Furthermore, the inclusion of start and end dates provides the complete visibility needed to manage the inspection timeline effectively. This comprehensive view promotes transparency and ensures that teams remain well-coordinated throughout the entire inspection lifecycle.

Scenario 7 : Editing an Inspection Activity in CORE on **Figure 3.24 - 3.32**

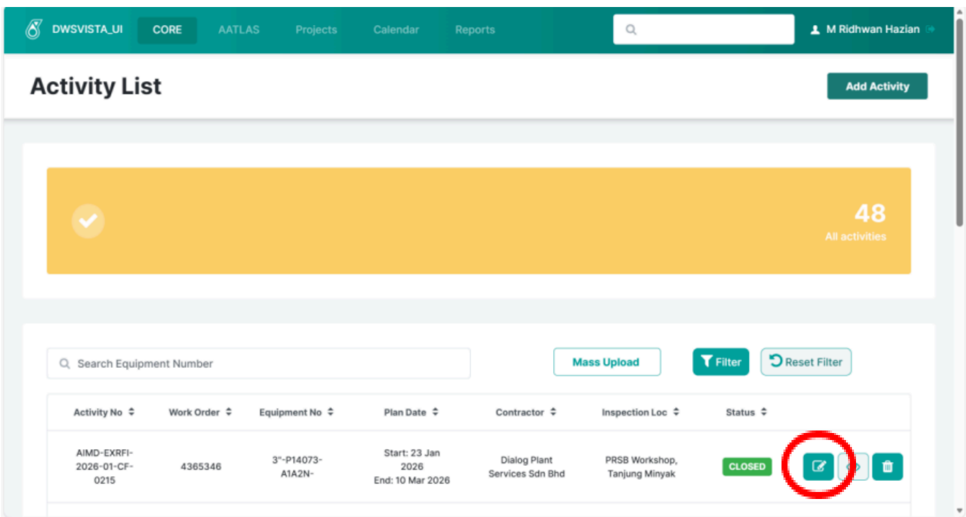


Figure 3.24 *: shows Edit Button on CORE module*

In the CORE module, users can modify an inspection activity by clicking the Edit button, an action that opens the configuration screen shown in **Figure 3.24**. This interface is uniquely organized using an accordion layout, which neatly categorizes each section according to specific phases or responsibilities within the inspection workflow. By adopting this design, the system enables users to focus on the specific sections relevant to the current activity status without being overwhelmed by unnecessary data. This structured approach maintains a clear and organized workspace, allowing for a more efficient process flow as the inspection progresses.

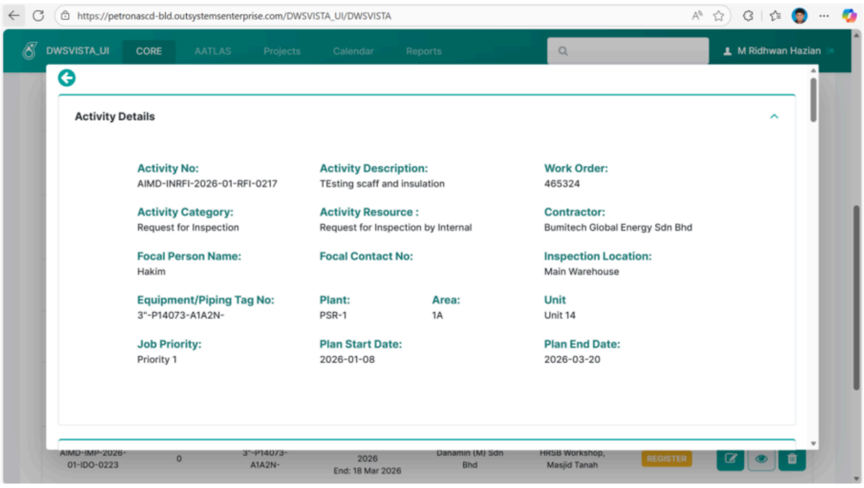


Figure 3.25 *: Shows Edit button on click Pop-Up Activity details accordion*

In the CORE module, the Activity Details accordion provides a snapshot of the current inspection, with the interface layout captured in **Figure 3.25**. This section aggregates key data such as the inspection type, equipment or piping references, and the assigned company or contractor. By design, this area is set to a read-only state, serving as a reliable reference point that allows users to review the registered information while safeguarding core details from accidental changes post-registration.

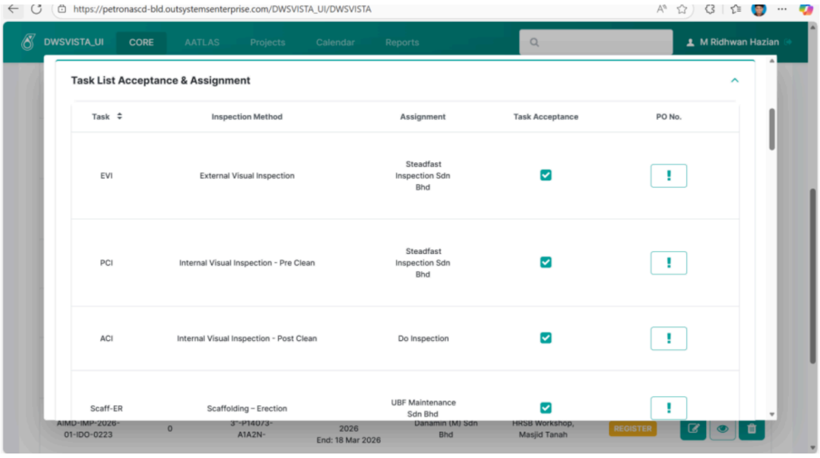


Figure 3.26 *: Shows Task List Acceptance & Assignment accordion*

Once an activity is added, its status is automatically set to Registered, which triggers the availability of the Task List Acceptance & Assignment accordion shown in **Figure 3.26**. Within this section, users assign contractors to each specific task associated with the inspection activity. While a Purchase Order (PO) number can be entered, it remains optional for each task. This step is critical to ensure that responsibilities are clearly defined and assigned before the execution phase begins.

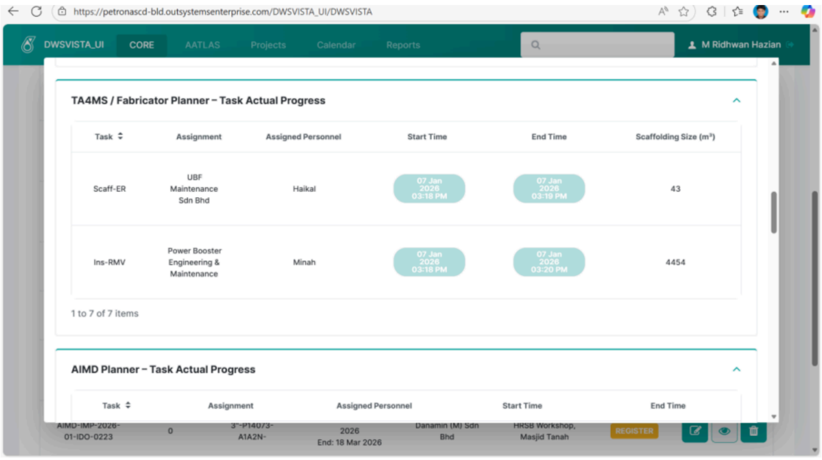


Figure 3.27 *: Shows TA4MS / Fabricator Planner – Task Actual Progress in Edit Pop-Up*

If the inspection activity includes Scaffolding or Insulation tasks, the TA4MS / Fabricator Planner - Task Actual Progress accordion appears, revealing the interface seen in **Figure 3.27**. In this section, TA4MS contractors are required to record the start and end dates for their assigned tasks. Following task completion, DI (Do Inspection) personnel must validate the work by entering specific details, such as scaffolding size, before the task can be officially marked as completed.

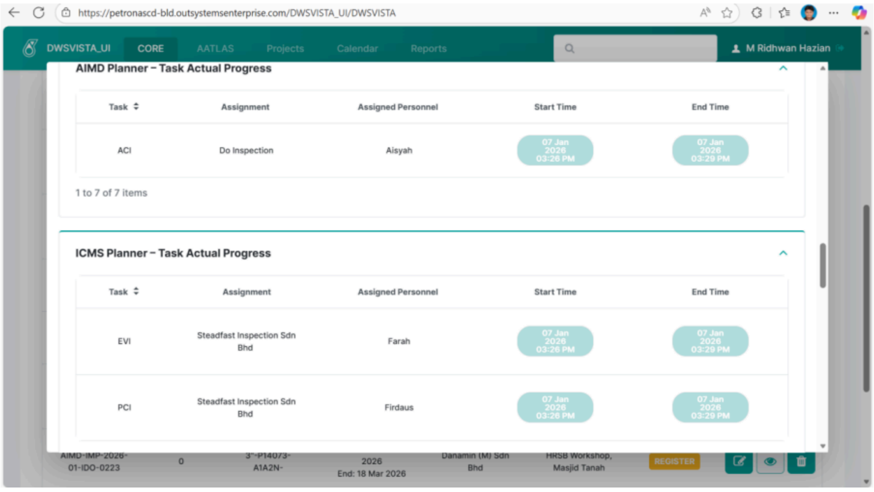
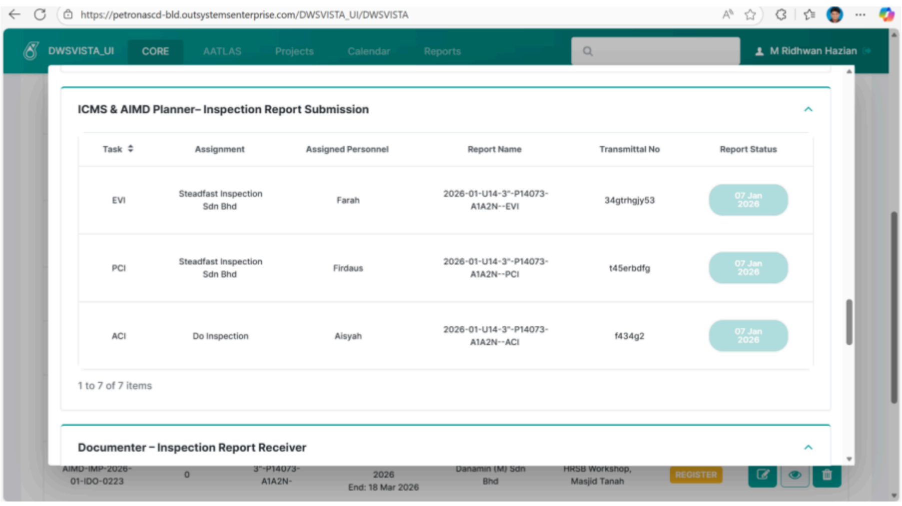


Figure 3.28 : *Shows AIMD Planner – Task Actual Progress and ICMS Planner – Task Actual Progress accordions*

For activities that do not involve scaffolding or insulation, the workflow bypasses the TA4MS section and proceeds directly to the AIMD Planner - Task Actual Progress and ICMS Planner – Task Actual Progress accordions, which follow the layout in **Figure 3.28**. Within these sections, both AIMD and ICMS personnel, along with their respective contractors, are required to record the start and end of each task. This requirement ensures that every activity is explicitly tracked within the system for accurate progress monitoring.



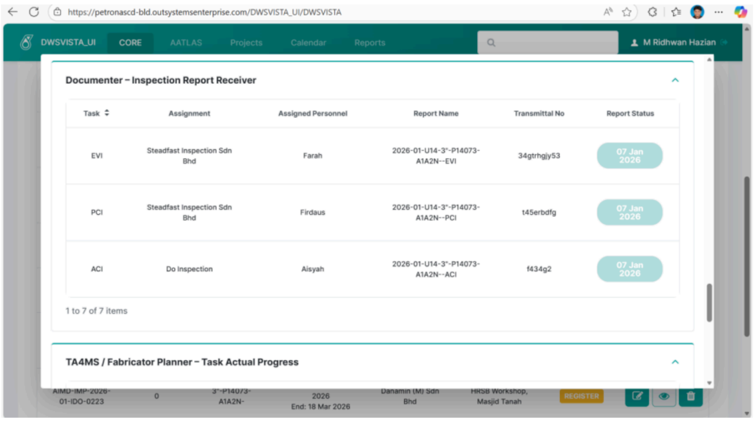


Figure 3.29 : ICMS & AIMD Planner – Inspection Report Submission and Documenter – Inspection Report Receiver accordions

Once all required Task Actual Progress entries are completed, the workflow advances to the ICMS & AIMD Planner - Inspection Report Submission accordion, an update that yields the interface seen in **Figure 3.29**. In this section, ICMS planners and DI personnel are responsible for submitting inspection reports for each completed task. Concurrently, upon report submission, the Documenter - Inspection Report Receiver accordion becomes available, enabling documenters to submit their confirmation once the reports have been reviewed.

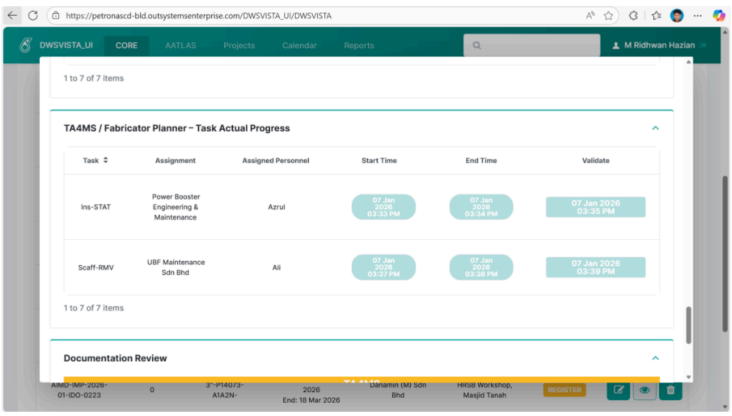


Figure 3.30 : Shows TA4MS – Task Actual Progress (Ins STAT and Scaff RMV) accordion

After all inspection reports are received and submitted, the workflow proceeds to the TA4MS - Task Actual Progress (Ins STAT and Scaff RMV) accordion, the layout of which is captured in **Figure 3.30**, provided these tasks are applicable. During this stage, insulation reinstatement or scaffolding removal tasks are executed; TA4MS contractors must record the start and end of these activities, while DI personnel perform the necessary validation before completion. If the activity does not include these specific tasks, the system automatically skips this accordion to maintain process efficiency.

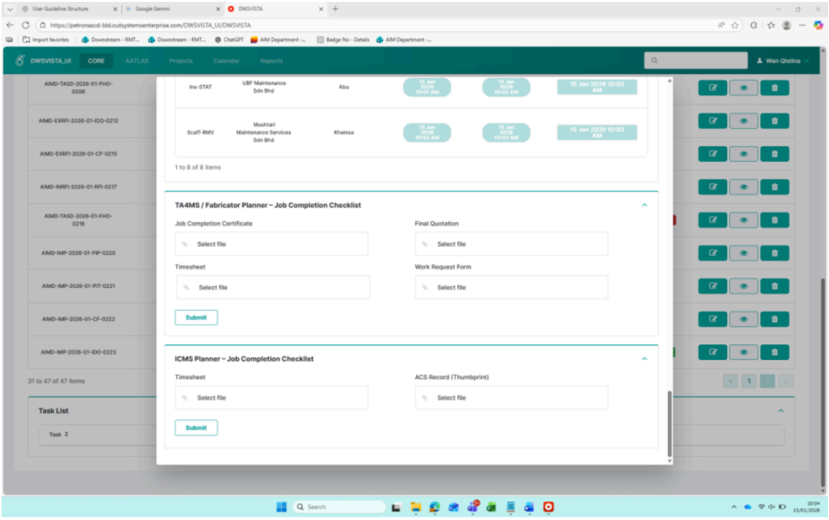
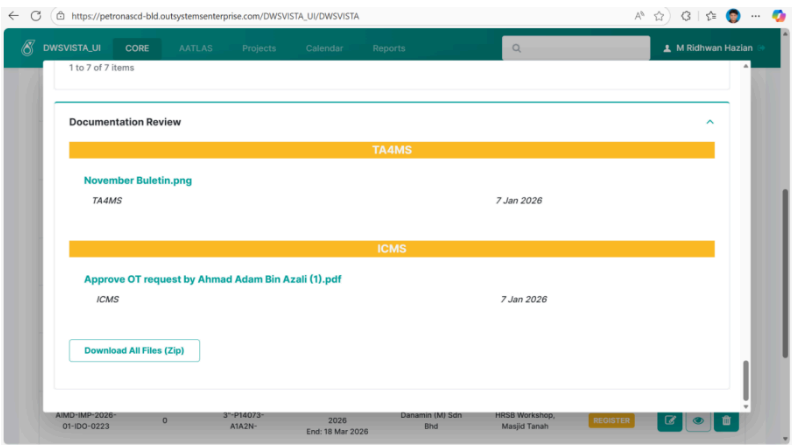
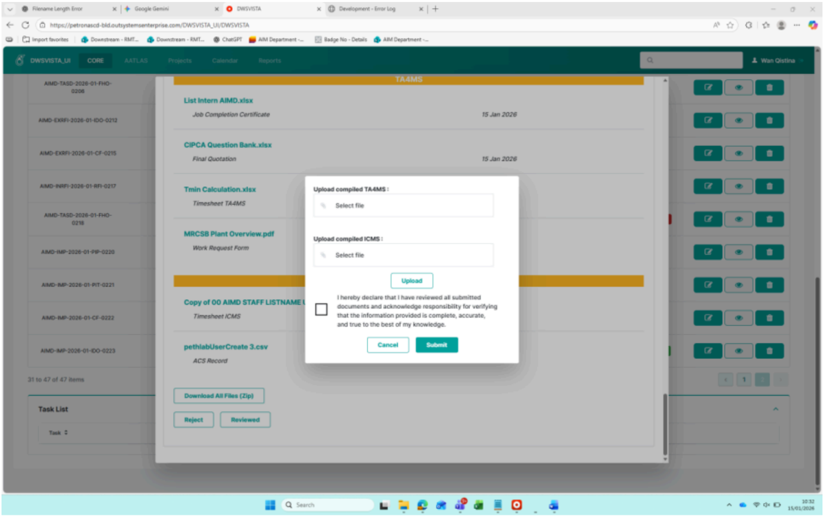


***Figure 3.31** : Shows Document Upload accordion*

Following task completion, the workflow advances to the Document Upload accordion, a transition reflected in **Figure 3.31**, where TA4MS (if applicable) and ICMS personnel are required to upload all supporting documents related to the inspection activity. Once the upload process is finalized, the system opens the Documentation Review accordion, allowing for the final verification of the submitted files.





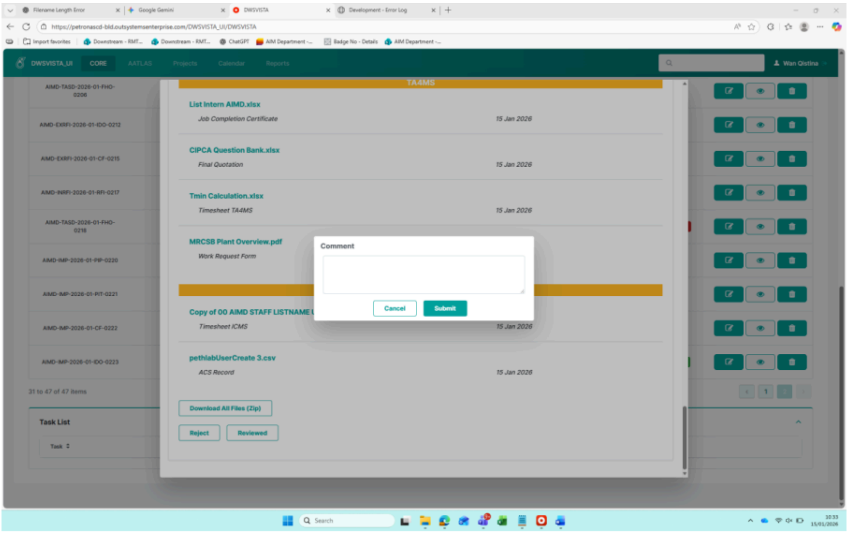


Figure 3.32 *: Shows Documentation Review accordion*

Within the Documentation Review accordion, the AIMD Planner performs a final review of all submitted documents, with the option to either approve or reject each file, the process is shown by the interface in **Figure 3.32**. If a document is rejected, the respective party is notified to re-upload a corrected version. Upon a successful review, the AIMD Planner uploads the finalized documents and submits the entire activity for HOD approval. At this final stage, the Head of Department evaluates the submission; any rejection requires a mandatory comment for clarification. Once HOD approval is granted, the inspection activity is officially marked as Closed in the system.

3.4.4 System Function

This section describes the internal operations and core functionalities performed by the system to support the execution of the defined use cases. System functions focus on how the system processes data, enforces business rules, and interacts with the database, rather than how users interact with the interface. These functions are mainly implemented at the back-end level to ensure data integrity, security, and reusability across the system. More logic(backends) can be found on **Appendix B**.

1. **User Identification and Session Management Function**

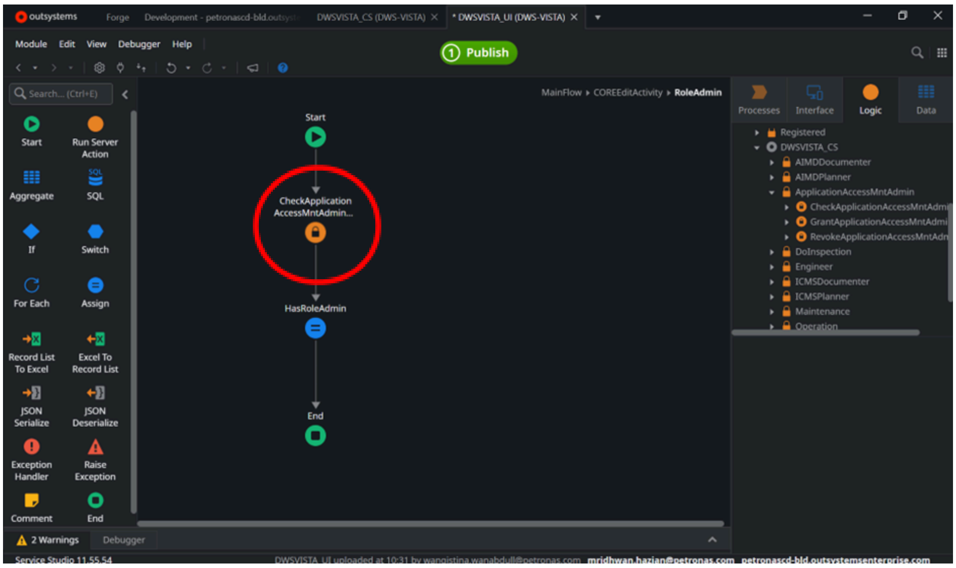


***Figure 3.33** : Shows example of user identification action*

This function is responsible for identifying the currently logged-in user, with the backend logic depicted in **Figure 3.33**. By retrieving the user's identity through the system's built-in authentication mechanism, the application associates all subsequent actions with that specific session. This process ensures that every operation is traceable and that data access is strictly controlled based on the authenticated user. Ultimately, this function serves as the critical foundation for all role-based and responsibility-based logic implemented throughout the system.

2. Data Validation and Business Rule Enforcement Function

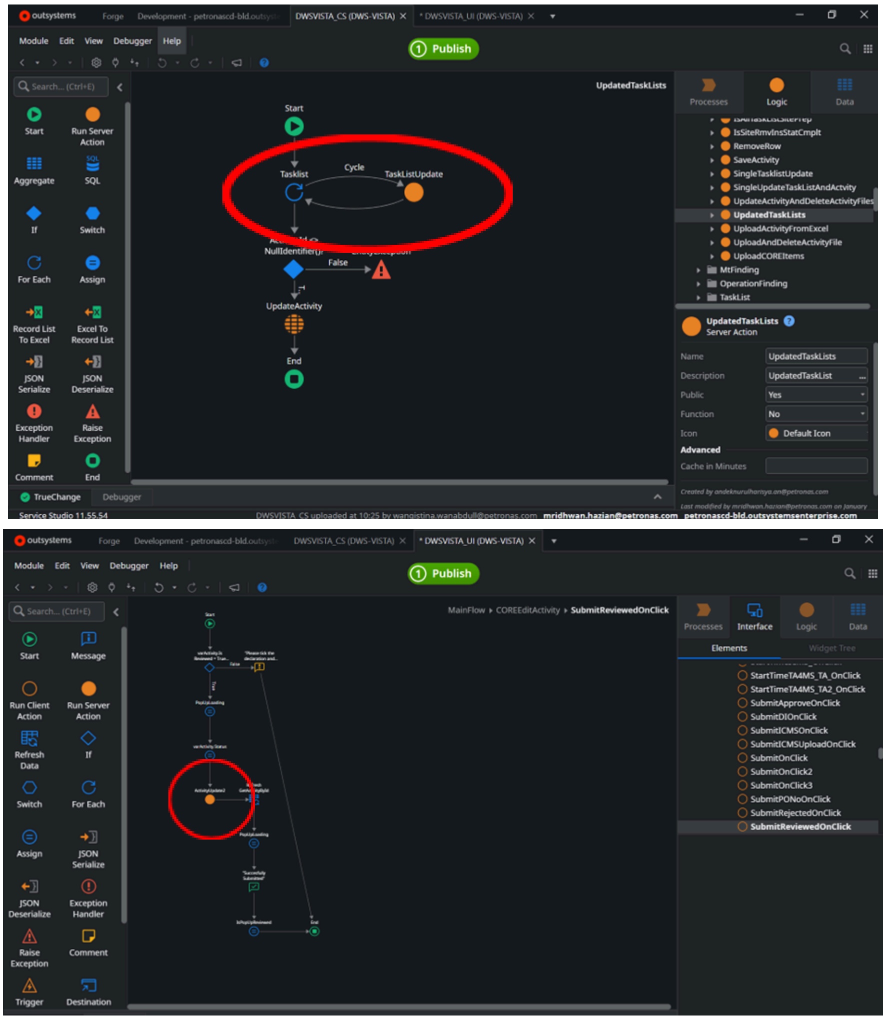


Figure 3.34 : Shows example of validation of input

This function validates all input data before it is processed or stored in the database, following the logic flow circled in **Figure 3.34**. It ensures that all mandatory fields are populated, data formats are correct, and specific business rules are strictly followed. For instance, the system verifies the existence of required identifiers, validates numeric ranges, and prevents the creation of duplicate or invalid entries. By performing these checks, the function plays a critical role in maintaining data consistency and safeguarding the system against errors caused by malformed input.

3. File Upload and Document Storage Function

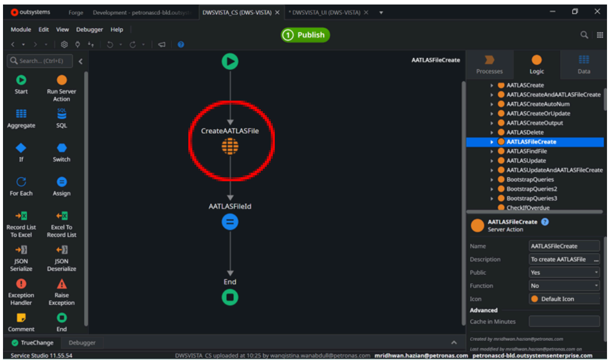


Figure 3.35 : Shows example of upload and store function

This function manages the uploading and storage of documents within the system, an operation defined by the backend action in **Figure 3.35**. It validates specific file attributes, including size, type, and content integrity, before saving the file to the database. Simultaneously, the system captures and stores essential metadata, such as the file name, upload timestamp, and associated record links. This ensures that all documents are securely archived and accurately mapped to their corresponding entities for easy retrieval.

4. Database Interaction and Data Persistence Function

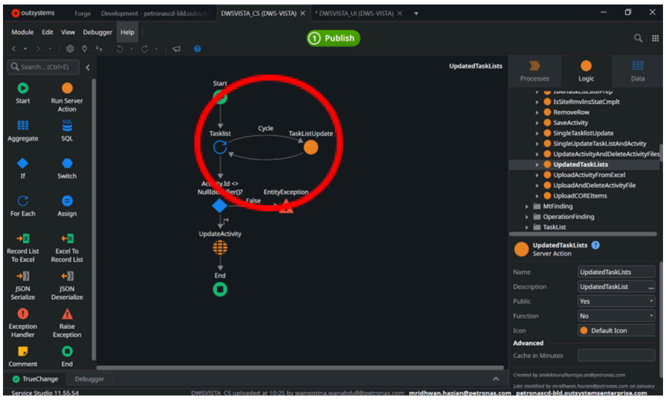


Figure 3.36 : Shows example of Data updating and insertion

This function handles all database operations, including data insertion, updates, and retrieval, with the underlying logic structured as seen in **Figure 3.36**. It ensures that records are stored accurately and that relationships between entities remain intact through valid foreign key references. Additionally, the function manages transactional integrity to prevent partial updates, ensuring consistent system behavior.

5. Error Handling and Exception Management Function

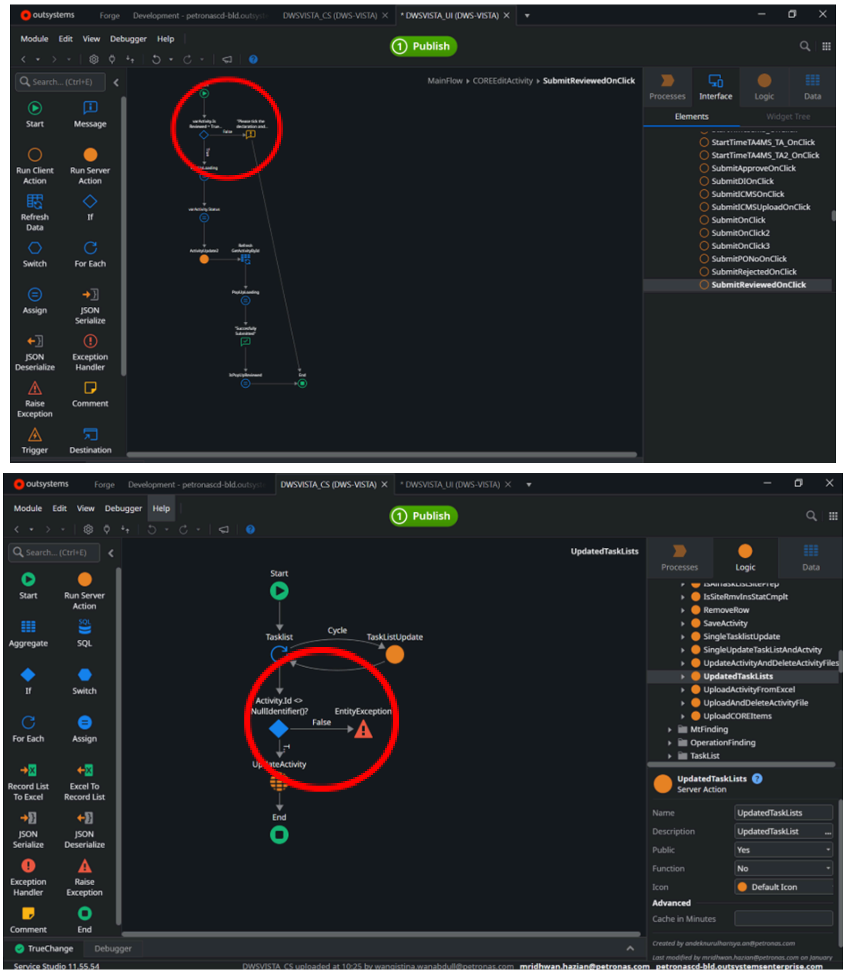


Figure 3.37 : Shows example of Detecting system errors

This function detects and manages system errors that may occur during execution, as represented in the logic flow of **Figure 3.37**. When an exception is encountered, such as a database constraint violation or an invalid operation, the system automatically halts incomplete transactions to protect data integrity and logs the error for monitoring. By intercepting these failures, the function significantly improves system reliability and assists administrators in efficiently identifying and resolving underlying issues.

6. Data Retrieval and Reporting Support Function

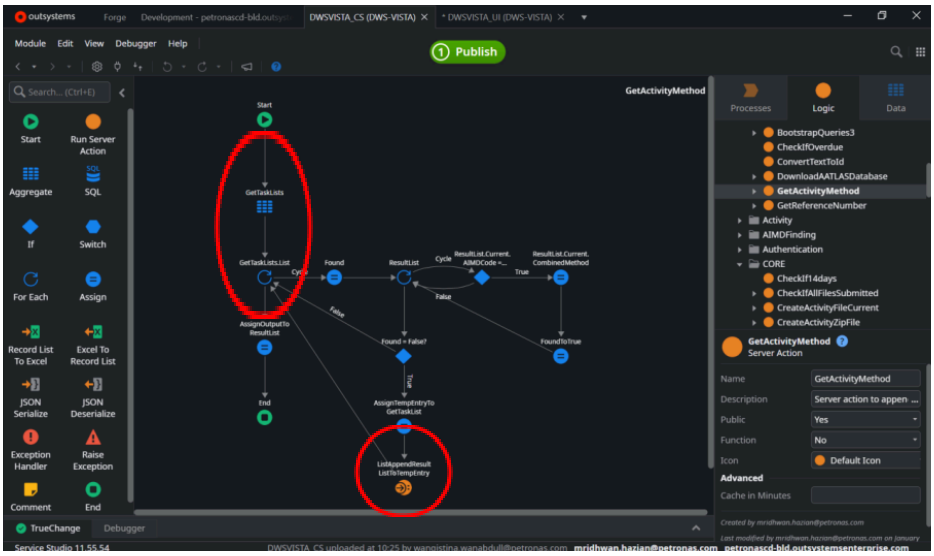


Figure 3.38 : Shows example of structured data turned into list

This function retrieves structured data from the database to support system reporting and data presentation, as shown in **Figure 3.38**. It enables the system to generate accurate lists, summaries, and records based on defined criteria, such as user area, equipment status, or uploaded documents. By ensuring that all displayed information reflects the real-time state of the system, this function provides the essential data necessary to support informed decision-making.

7. System Reusability and Modularity Function

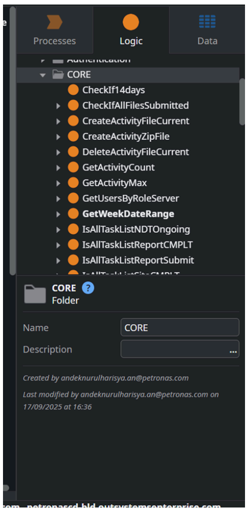


Figure 3.39 *: Shows example of system reusability where server action can be reused anywhere*

This function ensures that core logic is implemented in a modular and reusable manner, as depicted in the modular logic structure of **Figure 3.39**. By centralizing business logic within server-side functions, the system allows for seamless reuse across multiple screens, processes, and future integrations. This architectural approach significantly improves maintainability, reduces code redundancy, and provides the necessary scalability to support evolving system requirements.

Summary

In conclusion, the system functions form the backbone of the application by handling authentication, data processing, validation, file management, and database interactions. These functions operate primarily at the back-end level and are designed to support multiple use cases while maintaining security, consistency, and performance. By separating system functions from user interface logic, the system achieves a clear architectural structure that enhances maintainability and supports future enhancements.

CHAPTER 4: OTHER TASKS

4.1 Power Apps

4.1.1 Overview of Microsoft Power Apps

Microsoft Power Apps is a low-code application development platform that enables organisations to design and deploy internal applications quickly without extensive programming knowledge. It is commonly used to digitise operational processes, replace manual forms, and streamline data collection. Power Apps supports rapid development and integrates well with Microsoft services such as Excel, SharePoint, Outlook, and Power Automate.

In AIMD, Power Apps was adopted during the early phase of digitalisation as a fast and practical solution to address immediate operational needs. At that time, many inspection-related activities were still handled manually or through multiple Excel spreadsheets, which limited visibility and made tracking difficult.

4.1.2 AATLAS as a Power Apps-Based System

AATLAS, which stands for AIMD Action Tracking List and Assurance System, was originally developed using Microsoft Power Apps. The primary objective of AATLAS was to support AIMD in tracking inspection-related actions, monitoring assurance activities, and ensuring that assigned tasks were completed within the required timeline.

As implied by its name, AATLAS focused specifically on action tracking and assurance rather than full inspection scheduling, reporting, or analytics. Users were able to submit updates related to completed actions, while responsible personnel could monitor task progress through structured lists and status indicators.

At its initial stage, Power Apps provided a suitable platform for AATLAS due to its ease of development and accessibility, particularly for users who required a simple and mobile-friendly interface. A sample Power Apps–based AATLAS dashboard is shown in **Figure 4.1.**

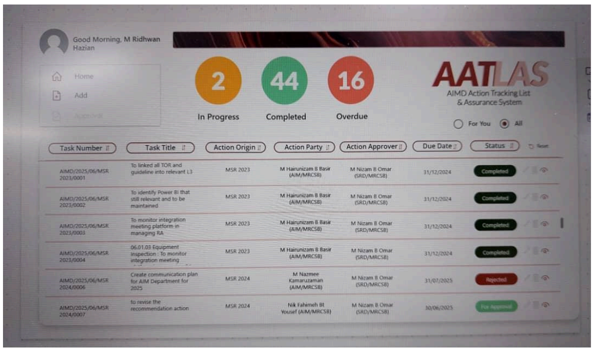


***Figure 4.1** : Shows AATLAS Power Apps main page*

4.1.3 Limitations of the Original Power Apps Implementation

Despite its advantages, the Power Apps implementation of AATLAS presented some limitations as usage increased. A primary challenge was its heavy reliance on Power Automate to track changes within the SharePoint database. For every update—such as a modification in a row or column—Power Automate needed to detect the change, send email notifications, and update statuses accordingly. This dependency added complexity to workflow management and introduced potential delays or errors if the automated flows did not execute as expected.

Additionally, managing complex workflows entirely within Power Apps became increasingly difficult as the volume of inspection-related actions grew. Performance and scalability concerns were observed, especially when multiple users accessed the system simultaneously.

These limitations highlighted the need for a more integrated and robust solution that could streamline workflows, ensure reliable automation, and provide real-time tracking and notifications without over-reliance on separate automation flows.

4.1.4 Integration of Power Apps with Power Automate in AATLAS

In the AATLAS system, Microsoft Power Apps and Power Automate were used together to support action tracking and assurance workflows. Power Apps functioned as the user interface, allowing AIMD users to submit updates, change action statuses, or trigger specific processes through form inputs and button actions.

Power Automate was integrated with Power Apps to handle background automation processes. For example, when a user clicked a button in Power Apps to mark an action as completed, Power Automate was triggered to send notification emails to relevant stakeholders. In addition, Power Automate monitored changes in the underlying data source, such as updates in Excel or database records. When an item was modified, automated flows were activated to send emails, update status fields, or log assurance activities.

This combination allowed AATLAS to automate repetitive tasks, reduce manual email handling, and improve visibility of action completion. However, as the system grew, managing multiple flows and data sources became more complex, especially when several Excel files were involved. This highlighted the limitations of relying on loosely connected platforms for long-term scalability.

4.1.5 Enhancement of AATLAS Using Power Apps

As part of the practical training, enhancements were implemented in the AATLAS Power Apps application to improve flexibility and control in managing action due dates. One of the key improvements was the introduction of an “Extend Due Date” feature for action parties.

This feature allowed an action party to request a due date extension by clicking a dedicated button within the Power Apps interface. Upon clicking the button, a popup form was displayed, prompting the user to provide a new proposed due date and justification for the extension. To maintain process discipline and prevent repeated delays, the system was designed to allow the due date to be extended only once for each action item.

Once the extension request was submitted, an automated email notification was sent to the designated action approver. The email was generated through Power Apps and provided clear details of the requested extension, including the original due date, the proposed new date, and the reason for the request.

In addition, a dedicated container was developed within the Power Apps interface for action approvers. This container displayed pending extension requests and allowed approvers to approve or reject the request directly within the application. The approval decision was then reflected in the system status and communicated to the action party through email notification.

To support this new functionality, several new columns were added to the AIMD database. These columns were used to store extension-related information such as extension status, extended due date, approval decision, and indicators to ensure that each action item could only be extended once. This ensured data consistency and allowed proper tracking of extension history.

Overall, this enhancement improved transparency, accountability, and efficiency in action tracking within AATLAS, while reducing manual email communication and ad-hoc follow-ups.

4.2 Power Automate

4.2.1 Overview of Microsoft Power Automate

Microsoft Power Automate is a workflow automation platform that enables users to automate repetitive tasks and business processes across various applications and services. It is commonly used to trigger actions based on events such as data changes, user inputs, or scheduled times. Power Automate integrates seamlessly with Microsoft tools including Power Apps, Outlook, Excel, and SharePoint, making it suitable for automating notification, approval, and data processing workflows.

In AIMD, Power Automate played a crucial role in reducing manual work related to inspection tracking and assurance activities by automating email notifications and status updates.

4.2.2 Role of Power Automate in AATLAS

Within the AATLAS system, Power Automate was used as the backend automation engine that worked in conjunction with Power Apps. While Power Apps served as the user interface for action tracking, Power Automate handled event-driven processes such as email notifications, approval routing, and status synchronization.

Power Automate flows were designed to monitor changes in the AIMD database or Excel-based data sources. When an action item was updated, marked as completed, or when a due date extension was requested, Power Automate automatically triggered the relevant workflow without requiring manual intervention.

This integration improved the responsiveness of the system and ensured that stakeholders received timely updates.

4.2.3 Email Notification and Approval Workflows

One of the primary uses of Power Automate in AATLAS was the automation of email notifications. Flows were configured to send emails to action parties, action approvers, and relevant stakeholders when specific conditions were met.

For example, when an action party submitted a due date extension request through Power Apps, Power Automate detected the data change and automatically sent an email to the action approver with the relevant details. Similarly, when an approver approved or rejected the request, another automated email was sent to inform the action party of the decision.

This automation eliminated the need for manual follow-ups and ensured that all communication was consistent and traceable.

4.2.4 Data Change Monitoring and Workflow Triggers

Power Automate was also used to monitor changes in the underlying data source used by AATLAS. Triggers were set up to detect when a record was created or modified, such as updates to action status, due dates, or approval fields.

Once a change was detected, Power Automate executed predefined actions such as updating related records, sending notification emails, or logging assurance activities. This event-driven approach ensured that the system remained up to date in real time and reduced the risk of missed actions.

However, managing multiple flows connected to various Excel files required careful coordination to avoid duplicate triggers or conflicting updates.

4.2.5 Contribution and Practical Experience

The primary focus of my work with Power Automate involved reviewing existing flows, understanding their trigger conditions, and assisting in improving the overall automation logic. One of my main contributions was the development of a flow for approving and rejecting due date extension requests. This flow automatically sends emails to action parties with clear status updates, using a professional and user-friendly design with CSS formatting to ensure readability and consistency for stakeholders.

In addition, another flow was developed to support operational tasks in SharePoint. This automation handled file copying and the creation of new PowerPoint folders—tasks that were previously performed manually. As a result, errors were reduced, significant time was saved, and overall operational efficiency was improved.

These experiences provided valuable exposure to workflow automation, trigger design, and email formatting, highlighting the importance of balancing functionality with user experience. They also demonstrated the practical benefits of Power Automate in streamlining repetitive tasks and enhancing process efficiency.

The knowledge and skills gained from these activities reinforced key principles later applied in the VISTA system, particularly in designing automated processes that are reliable, user-friendly, and seamlessly integrated with existing databases and workflows. **Figure 4.2** shows an example of a Power Automate flow, while **Figure 4.3** presents one of its outputs**.**

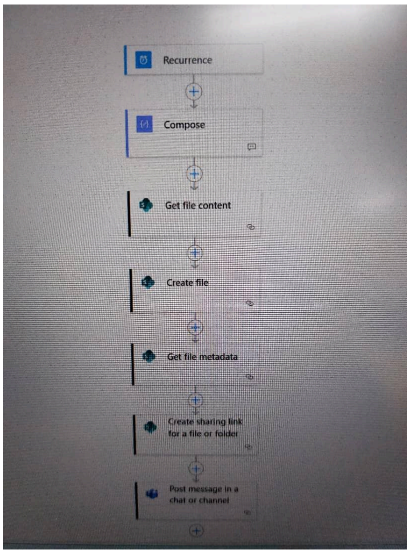


Figure 4.2 *: Shows example Power Automate flow.*

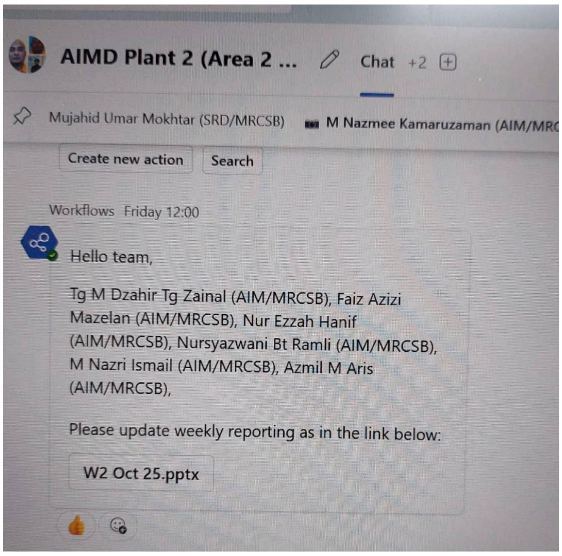


Figure 4.3 *: Shows Power Automate outputs to Microsoft Teams.*

CHAPTER 5: CONCLUSION AND RECOMMENDATION

5.1 Conclusion

Throughout the practical training period, the primary focus was on contributing to the development and enhancement of the VISTA (Visibility into Inspection Scheduling, Tracking & Assurance) system, particularly within the AATLAS and CORE modules. This experience provided hands-on exposure to OutSystems Service Studio, where a centralized and scalable digital platform was developed to address long-standing challenges in AIMD, such as fragmented data management, heavy reliance on multiple Excel files, and inefficient inspection activity tracking.

In addition to the main project, involvement in Power Apps and Power Automate tasks offered valuable insights into workflow automation and low-code application development. Enhancements made to the AATLAS module, including the implementation of the “Extend Due Date” feature, approval and rejection email automation with improved user-friendly design, and SharePoint-based file handling automation, highlighted the importance of aligning technical solutions with practical operational needs to improve overall efficiency.

Overall, this practical training experience provided meaningful exposure to real-world challenges related to data management and digital transformation within an organisational environment. It emphasised the importance of centralising data, automating repetitive processes, and designing intuitive workflows to enhance operational efficiency and visibility, particularly in a large organisation such as AIMD. Furthermore, the experience reinforced the significance of proper planning, analysis, and systematic implementation in developing reliable and scalable enterprise applications.

5.2 Recommendation

To AIMD:

1. Continue improving and centralising all inspection tracking and reporting systems, as multiple platforms and Excel files make data management inefficient.

2. Further utilise automation tools, such as Power Automate and OutSystems workflows, to reduce manual work and ensure timely notifications and approvals.

3. Enhance user-friendliness of the system, especially in email notifications and dashboards, to help staff adopt the platform more easily and reduce errors.

4. Maintain proper documentation and structured workflow logs to allow future enhancements or troubleshooting to be done more smoothly.

To UiTM:

1. Secure sufficient internship placements for students, as some students may find it difficult to obtain practical training opportunities.

2. Consider increasing the internship period to at least four months or more, so students have adequate time to learn, contribute, and complete meaningful projects.

APPENDIX A: Organizations and Locations

Figure A.1 *: Petronas Twin Tower*

Figure A.2 *: PSR - 1 (processing sweet crude) , MRCSB*

Figure A.3 *: PSR - 2 (processing sour crude), MRCSB*

Figure A.4 *: MG3 (lube base oil production), MRCSB*

Figure A.5 *: DELIMA (producing EURO-5 diesel), MRCSB*

Figure A.6 *: COGEN(power and steam generation), MRCSB*

Figure A.7 *: Non-Process Area work attire*

Figure A.8 *: Process Area Work Attire*

APPENDIX B: System Process Flow and Server-Side Logic (OutSystems)

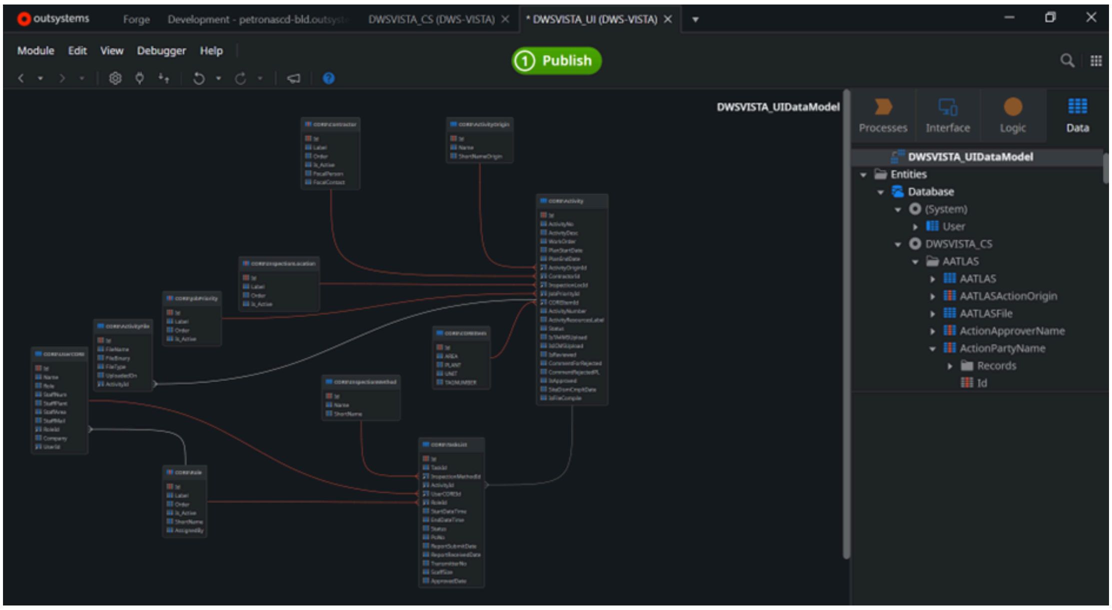


Figure B.1 : CORE Data Flow Diagram

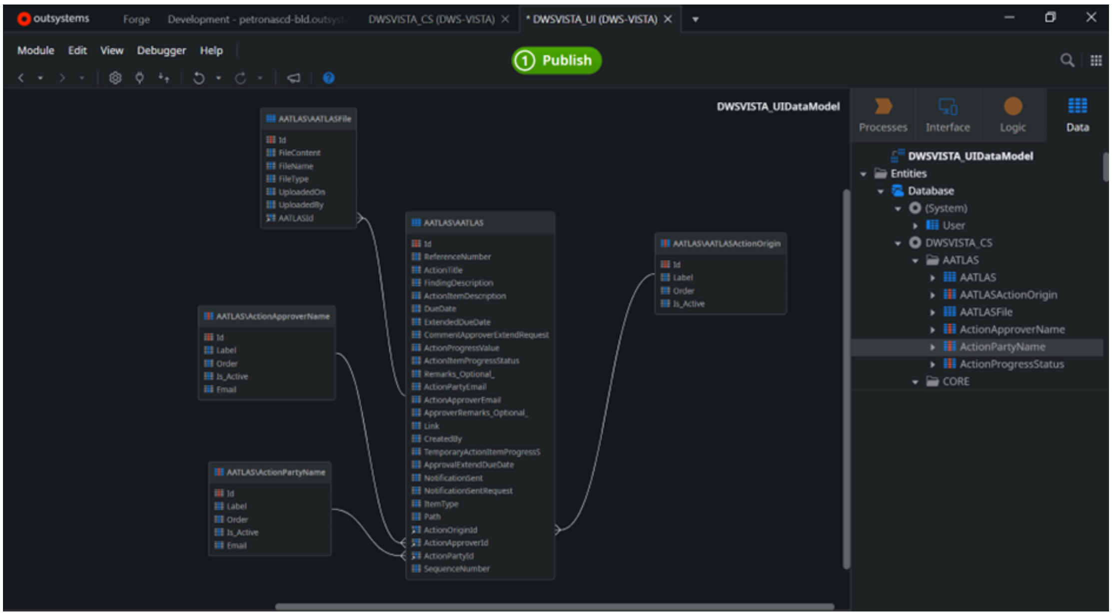
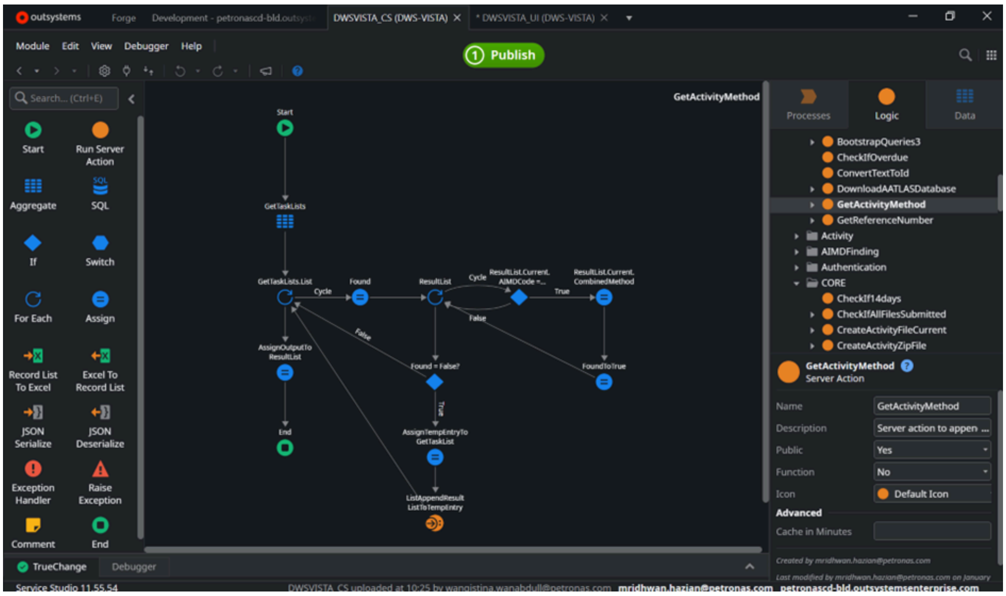
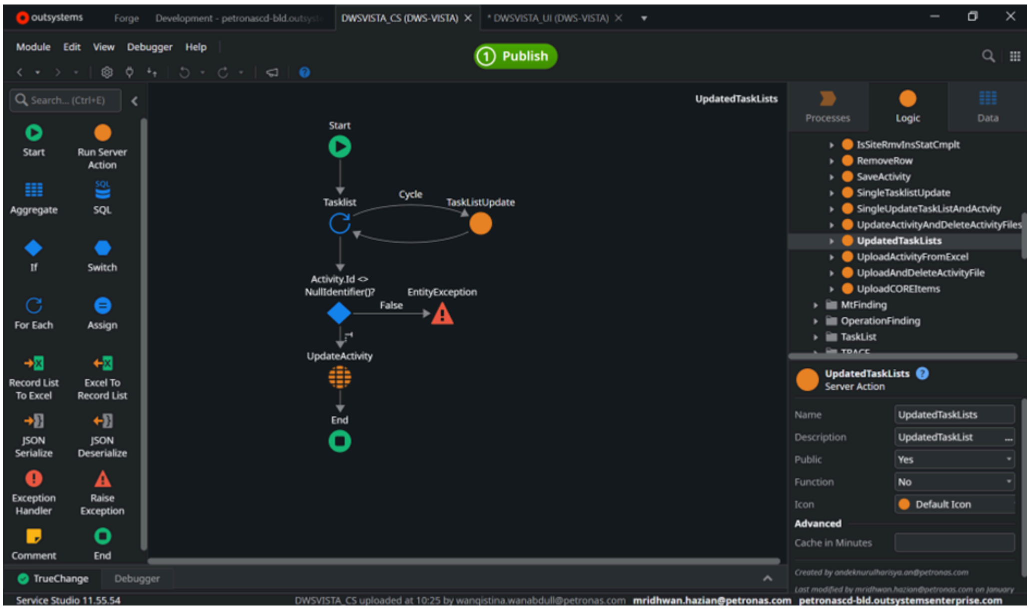
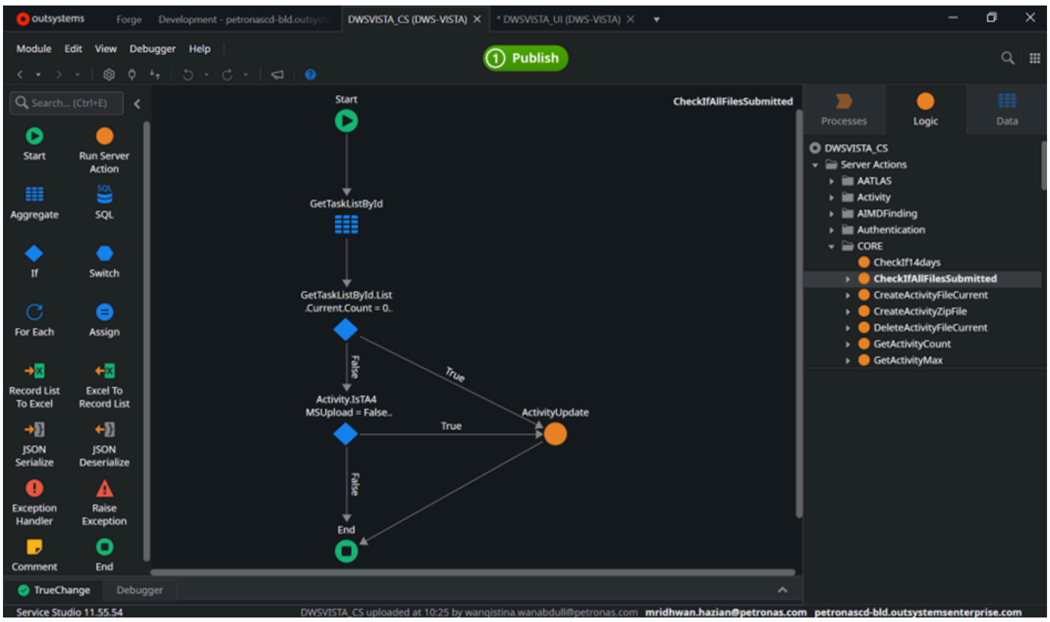


Figure B.2 : AATLAS Data Flow Diagram


outsystems
Forge
Development - petronascd-bld.outsys
DWSVISTA_CS (DWS-VISTA)
* DWSVISTA_UI (DWS-VISTA)
Module
Edit
View
Debugger
Help
Publish
GetActivityMethod
Processes
Logic
Data
Start
Run Server Action
Aggregate
SQL
If
Switch
For Each
Assign
Record List To Excel
Excel To Record List
JSON Serialize
JSON Deserialize
Exception Handler
Raise Exception
Comment
End
BootstrapQueries3
CheckIfOverdue
ConvertTextToId
DownloadAATLASDatabase
GetActivityMethod
GetReferenceNumber
Activity
AIMDFinding
Authentication
CORE
CheckIf14days
CheckIfAllFilesSubmitted
CreateActivityFileCurrent
CreateActivityZipFile
GetActivityMethod
Server Action
Name
Description
Public
Function
Icon
Advanced
Cache in Minutes
Yes
No
Default Icon
TrueChange
Debugger
Service Studio 11.55.54

outsystems
Forge
DWSVISTA_CS (DWS-VISTA)
* DWSVISTA_UI (DWS-VISTA)
Module
Edit
View
Debugger
Help
Publish
UpdatedTaskLists
Start
Tasklist
Cycle
TaskListUpdate
Activity.Id <> NullIdentifier()?
EntityException
False
UpdateActivity
End
Processes
Logic
Data
IsSiteRmvInsStatCmplt
RemoveRow
SaveActivity
SingleTasklistUpdate
SingleUpdateTaskListAndActvity
UpdateActivityAndDeleteActivityFiles
UpdatedTaskLists
UploadActivityFromExcel
UploadAndDeleteActivityFile
UploadCOREItems
MtFinding
OperationFinding
TaskList
UpdatedTaskLists
Server Action
Name
Description
Public
Function
Icon
UpdatedTaskList
Yes
No
Default Icon
Advanced
Cache in Minutes
Run Server Action
Aggregate
SQL
If
Switch
For Each
Assign
Record List To Excel
Excel To Record List
JSON Serialize
JSON Deserialize
Exception Handler
Raise Exception
Comment
TrueChange
Debugger
Service Studio 11.55.54

outsystems
Forge
DWSVISTA_CS (DWS-VISTA)
* DWSVISTA_UI (DWS-VISTA)
Module
Edit
View
Debugger
Help
Publish
CheckIfAllFilesSubmitted
Start
GetTaskListById
GetTaskListById.List.Current.Count = 0..
True
False
Activity.IsTA4 MSUpload = False..
True
ActivityUpdate
False
End
Processes
Logic
Data
DWSVISTA_CS
Server Actions
AATLAS
Activity
AIMDFinding
Authentication
CORE
CheckIf14days
CheckIfAllFilesSubmitted
CreateActivityFileCurrent
CreateActivityZipFile
DeleteActivityFileCurrent
GetActivityCount
GetActivityMax
Run Server Action
Aggregate
SQL
If
Switch
For Each
Assign
Record List To Excel
Excel To Record List
JSON Serialize
JSON Deserialize
Exception Handler
Raise Exception
Comment
TrueChange
Debugger
Service Studio 11.55.54

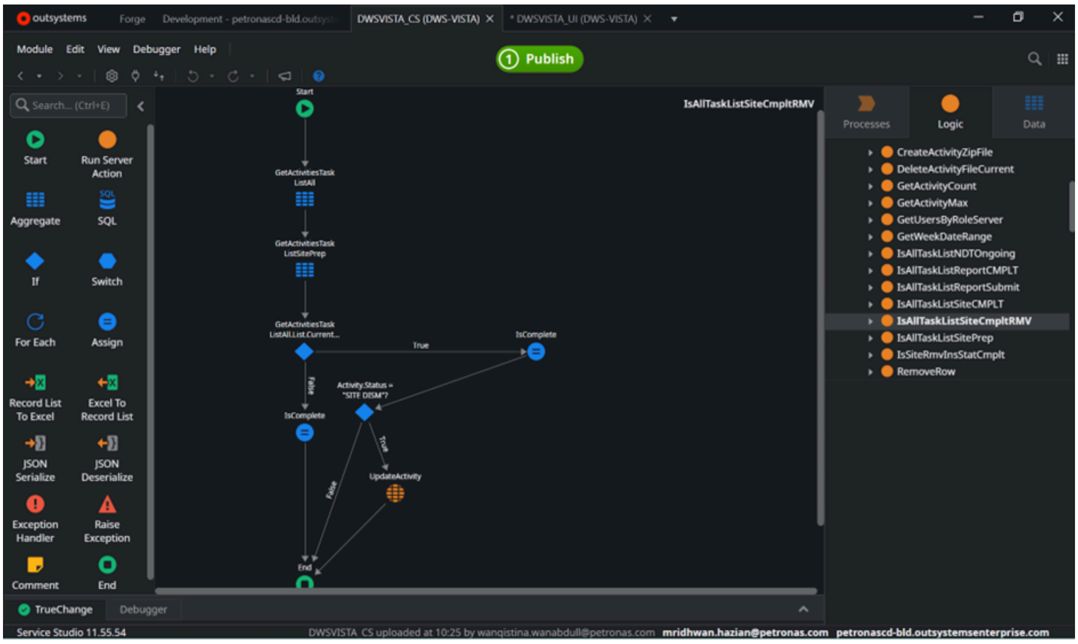


Figure B.3 : Example of server actions in Outsystem (Backends)